基于递推模型的板凳龙运行调控

摘　要

针对问题一，从等距螺旋方程出发，构建描述螺线上把手运动情况的 **微分方程模型，**求解得螺线位置参随时间变化关系。基于微元法和关联速度模型建立 **各把手位置参数与速度参数的递推模型，**使用 Newton-Raphson **方法** 进行求解。以 1s 为间隔对给定时间范围进行离散，得到 301 个时间点各把手的位置与速度，结果储存于表 result1.xlsx，发现对任意时间，螺线上沿极径增大方向把手速度递减。

针对问题二，将板凳抽象为矢量矩形，通过分离轴定理对矩形进行碰撞检测。构建以最大化盘入时间 t 为目标，所有矩形均不发生碰撞为约束条件的 **优化模型。**在给定时间范围内进行步长为 2s 的定步长搜索，在不同时间取值下对模型进行碰撞检测。利用约束条件缩小搜索的范围，反复减小步长进行搜索，最终得到满足约束条件的盘入时间最大值为 $\mathbf{412.50}\boldsymbol{s}$，此时舞龙队的位置和速度见文件 result2.xlsx。

针对问题三，本文建立了两个优化模型。首先不考虑时间 t 的变化，建立以龙头前把手与盘入点重合时龙身不发生碰撞为约束条件，最小化螺距为目标的 **静态优化模型。**与问题二类似，以精度 0.01m 循环遍历解得 $\mathbf{0.420}\boldsymbol{m}$。在静态模型的基础上考虑时间 t 的变化进一步建立以行进间龙身不发生碰撞为约束条件，最小化螺距为目标的 **动态优化模型，**最小螺距为 $\mathbf{0.448}\boldsymbol{m}$。

我们进行了 **模型优化，**综合考量结果准确性与计算量，取多组精度计算发现最优精度为 0.01m。对结果进行 **误差分析，**发现两种模型有 2cm 左右系统误差，推测由于静态模型未考虑龙头前把手到达 B 点前就已经发生第一次碰撞导致。

针对问题四，结合圆的对称性建立几何模型证明得调头曲线长度为定值，**即无法通过调整圆弧使得调头曲线变短。**将问题一单一盘入螺线行进路径拓展为圆弧段调头曲线与盘入盘出螺线，建立位置参数与速度参数分段递推函数模型。与问题一类似，运用 Newton-Raphson 方法求解，以 1s 为间隔对给定时间范围进行离散，得到 201 个时间点各把手的位置与速度，结果储存于表 result4.xlsx

针对问题五，证明得螺线上沿极径增大方向把手速度递减，再根据把手运动规律初步锁定最值情况。以最大化龙头速度为优化目标，舞龙队各把手的速度不超过 2m/s 为约束条件建立 **优化模型**。与问题二类似，首先使用区域搜索算法初步确定舞龙队取得最大速度时时间取值范围。通过循环遍历找到精确时间，此时龙身行进速度最大为 $2m/s$，龙头速度为 1.58m/s。

关键词：螺线 优化 微分方程 碰撞检测 Newton-Raphson **方法**

1 问题重述

1.1 题目背景

“板凳龙”是一个传统的舞龙活动，涉及将多个板凳连接在一起形成龙形结构。每节板凳有固定的长度、宽度和孔径。舞龙队沿一个等距螺线盘入时，龙头带领整个队伍行进，龙身和龙尾也随之移动。

1.2 求解问题

(1) 位置和速度计算：当龙头前把手的行进速度为 1 m/s，螺线的螺距为 55 cm 时计算舞龙队在沿螺线盘入过程中每秒的各个把手的位置和速度。

(2) 终止时刻的确定：确定舞龙队不能再继续盘入的时间点，并给出这一时刻各个把手的位置和速度。

(3) 调头空间的最小螺距：计算最小螺距，确保龙头前把手可以进入一个直径为 9 米的圆形调头空间。

(4) 调头路径优化：设计调头路径（由两段圆弧相切连接而成的 S 形曲线），并检查是否可以调整圆弧使调头路径更短。

(5) 最大行进速度的确定：以问题四的路径前进时计算龙头的最大行进速度，使得舞龙队的各个把手速度不超过 2 m/s。

2 问题分析

2.1 问题一的分析

问题一实则为递推模型。由于各把手都在螺线上，可建立微分方程模型描述把手的运动。而在舞龙过程中，前一个板凳带动后一个板凳运动，由几何关系可构建出前后把手位置间的递推关系。用微元法和关联速度模型可以建立前后把手速度间的递推关系。最后用迭代法解递推关系可得各个把手任意时刻的位置和速度。

2.2 问题二的分析

问题二本质是一个以最小化盘入时间为目标，所有板凳不发生碰撞为约束条件的优化问题。重点在于碰撞的检测，我们首先可抽象出出板凳的矩形模型，接着根据分离定理（SAT）对矩形进行碰撞检测。最后采用区域搜索算法初步确定最小时间范围，接着通过循环遍历找到所有满足条件的时间，根据优化目标求得最终结果。此时舞龙队的位置和速度可由问题一模型求解。

2.3 问题三的分析

问题三本质是一个以最大化螺距为目标，所有板凳不发生碰撞为约束条件的优化问题。思路有两条，其一为静态角度出发，考虑龙头前把手刚进入调头边界时所有板凳不发生碰撞。其二为动态角度出发，考虑舞龙队伍盘入调头边界的全过程。两种模型最后均回归到优化模型的求解，使用问题二中的区域搜索法求解。

2.4 问题四的分析

问题四首先可结合圆的对称性质和切线的性质求解调头曲线的长度。把手位置和速度的求解在前三问的基础上将单一盘入螺线改为四段曲线。我们分类求出位置、速度的递推关系，并迭代求解（同问题一）。

2.5 问题五的分析

问题五本质上是一个以最大化龙头速度为目标，舞龙队各把手速度不超过 2m/s 为约束条件的优化模型。首先可根据问题四求解结果初步判定舞龙队最大速度情况，接着

运用全域搜索法遍历时间，求得舞龙队运动过程中的最大速度。令该速度等于 2m/s，再将其作为初始值代入速度递推公式求得最大龙头速度。

3 模型假设

由于机器臂真实结构较为复杂，对机器臂系统进行如下处理：

1. 假设板凳在运动过程中不会发生形变。

2. 假设舞龙队在开始盘入螺线的初始状态以垂直于极轴竖直排列于 x=16p 处。

4 符号说明

表 1 列出了本文需要的符号，文中出现的其它符号将在出现时进行解释。

Table 1: 符号说明

序号	符号	符号描述
1	O	极点
2	A	龙头初始位置
3	B	龙头盘入与调头区边界相交位置
4	C_1, C_2	调头圆弧
5	a, b, c	直线方程系数
6	P_i	第 i 节龙身前把手
7	α	P 处螺线切线与极径垂线夹角
8	β	P 处板凳所在直线与极径垂线的夹角
9	θ	盘入螺线绕过的角度，方向与极角相同
10	d	板凳前后把手间距离
11	θ^{param}	曲线参数方程所对应参数
12	v	把手线速度
13	d	板凳前后把手间距离
14	m	曲线上点垂直极径的基准线
15	l	板凳的长
16	h	板凳的宽

5 模型建立与求解

5.1 问题一：位置和速度计算

5.1.1 模型的准备

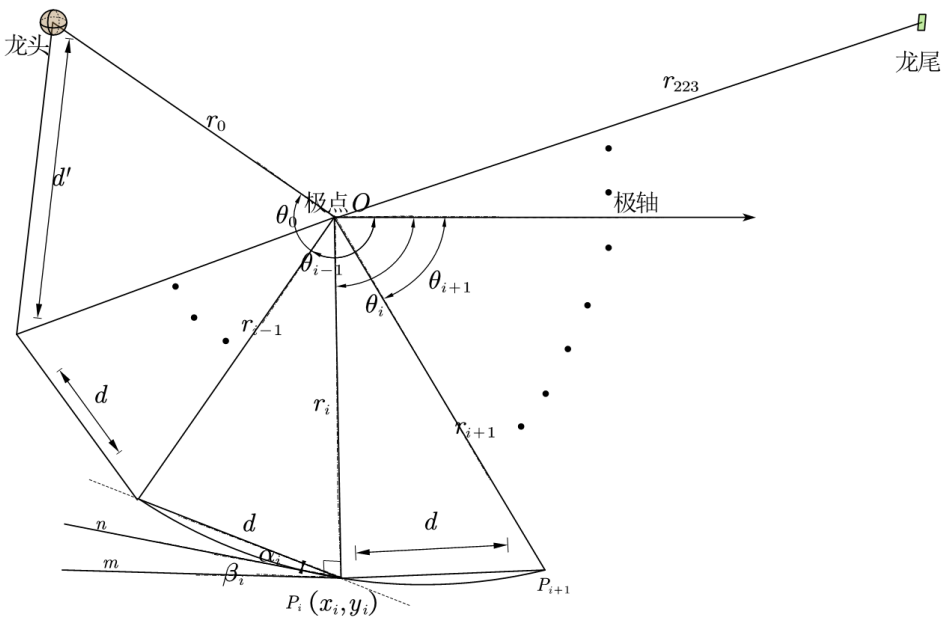

Figure 1: 盘入示意图

把手运动主要控制参数 $(\alpha, \beta, \theta, (x, y), d)$。

其中，i 表示第 i 个把手（i=0,1,...,223）；α 为 P 处螺线切线与极径垂线夹角；β 为 P 处板凳所在直线与极径垂线的夹角；θ 为盘入螺线走过的角度，方向与极角相同；(x,y) 为把手坐标，d 为板凳前后把手间距离。

5.1.2 模型的建立

5.1.2.1 螺线模型的确立

舞龙队沿等距阿基米德螺线顺时针盘入，以螺线中心为极点 O，射线 OA 为极轴建立极坐标系，该螺线的极坐标方程为：

$$r = \frac{p}{2\pi}\theta + 16p, \theta < 0 \tag{1}$$

其中，θ 为龙头顺时针转过的角度（如图 2），随着时间 t 增加而变小，则极角为 $\theta + 32\pi$。

5.1.2.2 微分方程模型的建立

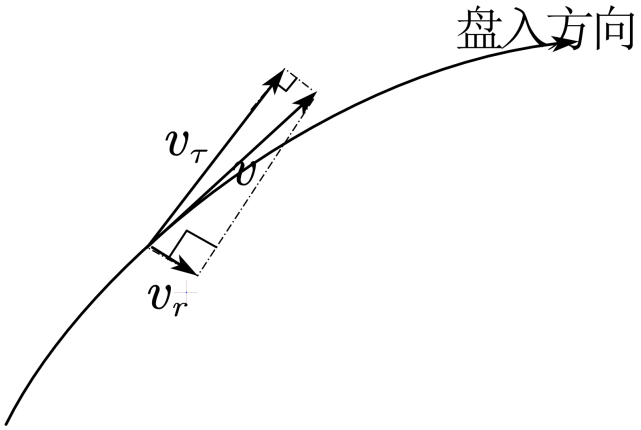

Figure 2: 速度分解示意图

现讨论龙头前把手的运动，则式 (1) 中 θ 为时间 t 和龙头前把手速度 v 的函数 $\theta(v,t)$。其中 v 由切向速度 v_τ 和径向速度 v_r 合成。首先求出切向速度：

$$v_\theta = r \cdot \omega = r \cdot \frac{d\theta}{dt} = r(\theta)\frac{d\theta}{dt} \tag{2}$$

接着求出径向速度：

$$v_r = \frac{dr}{dt} = r'(\theta)\frac{d\theta}{dt} \tag{3}$$

复合出龙头前把手的线速度：

$$v^2 = v_\tau^2 + v_r^2 \quad = (r^2(\theta) + r'(\theta)^2)(\frac{d\theta}{dt})^2 \tag{4}$$

两边同时开方得到微分方程，开方时注意 $d\theta/dt < 0$，其初值条件为：$\theta(t=0) = 32\pi$。**微分方程模型如下：**

$$\begin{cases} v = -\sqrt{r^2(\theta) + r'(\theta)^2} \cdot \dfrac{d\theta}{dt}, \\ \theta|_{t=0} = 32\pi, \end{cases} \tag{5}$$

特别地，第一问中 $v = 1m/s$，为定值。

5.1.2.3 各个把手的位置和速度递推模型的确立

把手位置参数递推公式 为方便距离的计算我们将阿基米德螺线的极坐标方程改写为直角坐标下的参数方程：

$$\begin{cases} x_i = \frac{p}{2\pi}(32\pi + \theta_i)cos(32\pi + \theta_i) \\ y_i = \frac{p}{2\pi}(32\pi + \theta_i)sin(32\pi + \theta_i) \end{cases} \tag{6}$$

t 时刻，相对于前一把手 $P_i(x_i(\theta_i), y_i(\theta_i))$，后一把手坐标 $P_{i+1}(x_{i+1}(\theta_{i+1}), y_{i+1}(\theta_{i+1}))$，有前后把手间的距离为 d：

$$\sqrt{(x_{i+1}-x_i)^2+(y_{i+1}-y_i)^2}=d, \tag{7}$$

可据此构建 θ_{i+1}、θ_i 间的递推公式。

为方便求解，下面定义 $\Delta\theta_i$：

$$\Delta\theta_i := \theta_{i+1}-\theta_i \tag{8}$$

把手速度递推公式 把手 P_{i+1} 的速度 v_{i+1} 也由 P_i 的速度 v_i 推导。

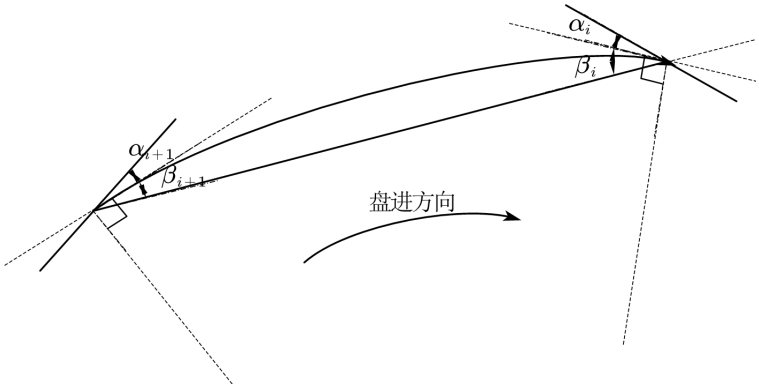

Figure 3: 关联速度模型

假设板凳在运动过程中不会发生形变，建立关联速度模型，分别将 v_i 与 v_{i+1} 沿杆、垂直于杆分解（图 3），得到：

$$v_i cos(\alpha_i-\beta_i)=v_{i+1}cos(\alpha_{i+1}-\beta_{i+1}) \tag{9}$$

其中，P_i 处螺线的切线与垂直极径的直线 m 的夹角为 α_i，P_iP_{i+1} 间的割线与垂直极径的直线 m 的夹角为 β_i。

如图 4，采用微元法求解 α、β，对于任意给定经过 P_i 的曲线 r_θ，在曲线上取距离 P_i 极近的 P_i'，有：

$$tan\phi=\frac{dr}{r\cdot d\theta}=\frac{\dot{r_\theta}}{r} \tag{10}$$

其中，$d\theta$ 为极角微元，dr 为极径微元，γ 为曲线的切线与垂直极径的直线 m 的夹角。

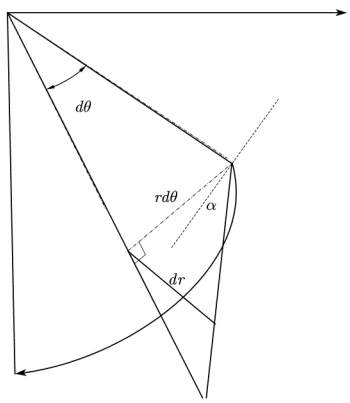

Figure 4: 微元模型

将螺线极坐标方程 (1) 代入式 (10)，得到螺线的切线与垂直极径的直线 m 的夹角 α：

$$\alpha = arctan\frac{\dot{r_{1\theta}}}{r} = arctan\frac{1}{\theta} \tag{11}$$

写出直线 $P_iP_{i+1} : ax + by + c = 0$ 的极坐标方程：

$$r = \frac{-c}{acos\theta + bsin\theta} \tag{12}$$

将该方程代入式 (10)，得到直线与垂直极径的直线 m 的夹角 β：

$$\beta = arctan\frac{\dot{r_{2\theta}}}{r} = arctan\frac{atan\theta - b}{a + btan\theta} \tag{13}$$

将 α 与 β 代入式 (9) 可得到 v_{i+1} 与 v_i 间的递推公式。

5.1.3 模型的求解

5.1.3.1 螺线模型的求解

将螺距 $p = 0.55m$ 代入式 (1)，得螺线方程：

$$r = \frac{0.55}{2\pi}\theta \tag{14}$$

5.1.3.2 微分方程的求解

先求微分方程通解，两边积分：

$$\int vdt = \int (r^2(\theta) + r'(\theta)^2)(\frac{d\theta}{dt})^2 \tag{15}$$

通解为：

$$\frac{1}{2}(r\sqrt{r^2+k^2}+k^2ln(r+\sqrt{r^2+k^2}))=-kv\cdot t+C \tag{16}$$

其中为 C 常数，$r=\frac{p}{2\pi}\theta+16p, k=\dfrac{p}{2\pi}$。将初值条件代入微分方程通解得特解：

$$\frac{1}{2}(r\sqrt{r^2+k^2}+k^2ln(r+\sqrt{r^2+k^2}))=-kv\cdot t, t\in[0,300] \tag{17}$$

将龙头前板凳恒定速率 $v=1m/s$ 代入特解（式 (17)），得到 t 时龙头前把手的位置，如图 5。

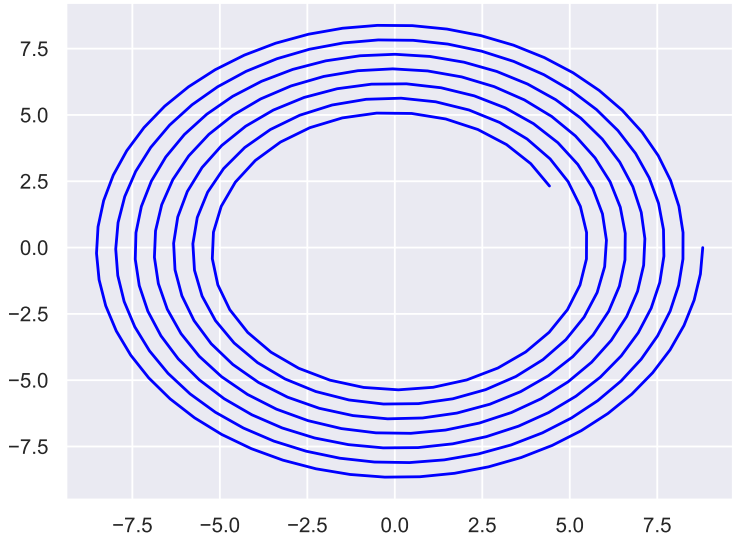

Figure 5: 前 300S 龙头前把手运动示意图（$\Delta t=1s$）

5.1.3.3 各个把手的位置和速度的求解——迭代法

将式 (6) 代入式 (7) 可得 θ_{i+1} 和 θ_i 的递推关系。初始值 $\theta_0=\theta$，θ 即为龙头前把手在 t 时刻位置所对应极角，满足式 (17)。

采用牛顿迭代法求解 $\Delta\theta_i$，将 $\Delta\theta_i$ 代入式 (8)，可求出每个把手对应 θ_i。将 θ_i 代入螺线参数方程 (6) 得到时间 t 时各个把手的位置。以等步长 $\Delta t=1s$ 在 [0,300s] 范围内离散时间变量 t，得到部分结果如表 2所示，为使表格能够放在论文中，保留两位小数。完整结果见文件 result1.xlsx。

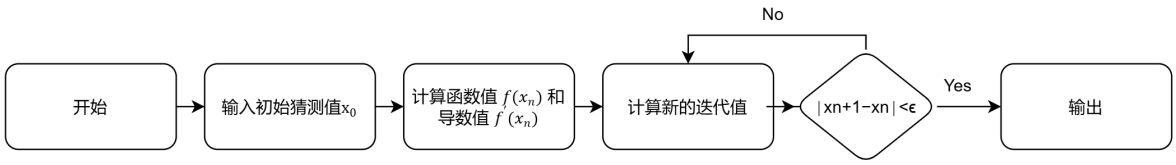

Figure 6: 牛顿迭代法流程图

Table 2: 把手位置（部分）

	0s	60s	120s	180s	240s	300s
龙头 x (m)	8.80	5.80	-4.08	-2.96	2.59	4.42
龙头 y (m)	-0.00	-5.77	-6.30	6.09	-5.36	2.32
第 1 节龙身 x (m)	8.80	7.46	-1.45	-5.24	4.82	2.46
第 1 节龙身 y (m)	2.86	-3.44	-7.41	4.36	-3.56	4.40
第 51 节龙身 x (m)	8.80	8.80	-5.54	2.89	6.00	-6.30
第 51 节龙身 y (m)	85.36	25.46	6.38	7.25	-3.83	0.47
第 101 节龙身 x (m)	8.80	8.80	8.80	1.90	-4.92	-6.24
第 101 节龙身 y (m)	167.86	107.96	48.08	-8.47	-6.38	3.94
第 151 节龙身 x (m)	8.80	8.80	8.80	8.80	8.80	7.04
第 151 节龙身 y (m)	250.36	190.46	130.58	70.71	10.89	4.39
第 201 节龙身 x (m)	8.80	8.80	8.80	8.80	8.80	8.80
第 201 节龙身 y (m)	332.86	272.96	213.08	153.21	93.39	33.63
龙尾（后）x (m)	8.80	8.80	8.80	8.80	8.80	8.80
龙尾（后）y (m)	367.51	307.61	247.73	187.86	128.04	68.28

同理，将初始值龙头前把手速度 $v_0 = 1m/s$ 代入递推式 (9)，运用牛顿迭代法解得各个把手的速度。部分结果如表 3所示，为在论文中显示完整表格，对数据保留五位小数处理。完整结果见文件 result1.xlsx。

Table 3: 把手速度（部分）

	0s	60s	120s	180s	240s	300s
龙头 (m/s)	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000
第 1 节龙身 (m/s)	0.99995	0.99999	0.99999	0.99998	0.99997	0.99995
第 51 节龙身 (m/s)	0.99995	0.99732	0.99986	0.99980	0.99968	0.99943
第 101 节龙身 (m/s)	0.99995	0.99732	0.99852	0.99968	0.99952	0.99918
第 151 节龙身 (m/s)	0.99995	0.99732	0.99852	0.99912	0.99743	0.99904
第 201 节龙身 (m/s)	0.99995	0.99732	0.99852	0.99912	0.99743	0.99634
龙尾（后）(m/s)	0.99995	0.99732	0.99852	0.99912	0.99743	0.99634

可见任意时间，螺线上沿极径增大方向把手速度递减。

5.2 问题二：终止时刻的确定

5.2.1 模型的准备

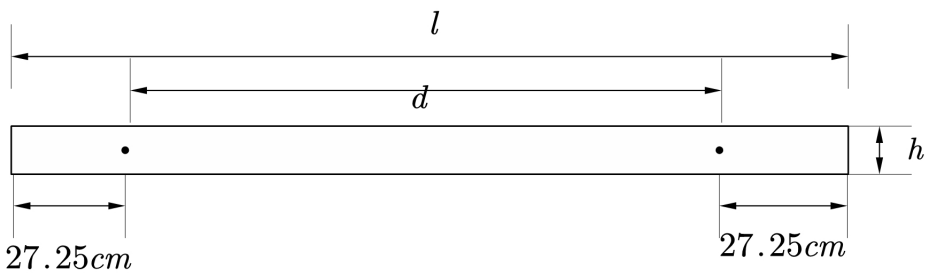

Figure 7: 板凳参数示意图

板凳主要控制参数 $(l, d, h, (x, y), (x', y'))$。

其中，l 为板凳长，d 为板凳前后把手间距离，h 为板凳的宽。(x,y) 为把手坐标，(x', y') 为矩形中轴线两端点坐标。

5.2.2 碰撞模型的建立

当板凳间碰撞发生时，舞龙队盘入过程终止，我们需要找到舞龙队不发生碰撞的最大盘入时间，即构建以 $max\ t$ 为目标的优化模型。

板凳碰撞矩形 现将板凳抽象为矩形，在平面中以一对互相垂直的向量表出矩形（图 8）。对于板凳 ABCD，其长边向量与向量 (x_2-x_1, y_2-y_1) 平行，其中 (x_1, y_1) 为第一个孔的坐标，(x_2, y_2) 为第二个孔的坐标。则长边向量 $\boldsymbol{s}=(x_l, y_l)=(x_2'-x_1', y_2'-y_1')$ 满足：

$$\begin{cases} x_1' = x_2 - k(x_2 - x_1) \\ y_1' = y_2 - k(y_2 - y_1) \end{cases} \qquad \begin{cases} x_2' = x_1 + k(x_2 - x_1) \\ y_2' = y_1 + k(y_2 - y_1) \end{cases} \tag{18}$$

短边向量 $\boldsymbol{v}$ 与长边向量垂直：

$$\boldsymbol{v} = \frac{h}{2}\left(\frac{-y_k}{\sqrt{x_k^2+y_k^2}}, \frac{x_k}{\sqrt{x_k^2+y_k^2}}\right) \tag{19}$$

则碰撞矩形的四个顶点：

$$(x_1', y_1') \pm \boldsymbol{v}, (x_2', y_2') \pm \boldsymbol{v} \tag{20}$$

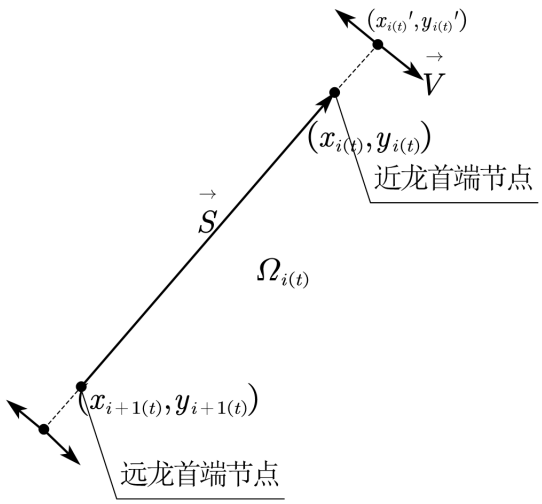

Figure 8: 板凳的矩形模型

定义 $\Omega_{(i,t)}$ 为时间 t 时，第 i 条龙身由四个顶点（式 (28)）构成的矩形区域。

5.2.2.1 优化模型的确立

当平面上任两个矩形出现重叠区域，则矩形产生碰撞。本问约束条件即为所有矩形不发生碰撞，用集合语言表示为：

$$\Omega_{(i,t)} \cap C_R\Omega_{(i,t)} = \emptyset, \tag{21}$$

其中，$\Omega_{(i,t)}$ 表示 t 时刻第 i 块板的二维空间区域。

结合时间最小目标建立优化模型：

$$
\max \quad t
$$

$$
s.t.\begin{cases}
\Omega_{(i,t)} \cap C_R\Omega_{(i,t)} = \emptyset \\
x(i,t=0) = 16d \\
y(i,t=0) = 0 \\
r = \dfrac{p}{2\pi}\theta + 16p, \theta < 0 \\
\begin{bmatrix} x_{(i,t)} \\ y_{(i,t)} \end{bmatrix} = r(\theta_t) \begin{bmatrix} cos\theta_t \\ sin\theta_t \end{bmatrix} \\
p = 0.55m \\
\frac{1}{2}(r\sqrt{r^2+k^2} + k^2 ln(r+\sqrt{r^2+k^2})) = -v \cdot t, t \in [0,300] \\
\sqrt{(x_{i+1}-x_i)^2 + (y_{i+1}-y_i)^2} = d
\end{cases} . \tag{22}
$$

5.2.3 模型的求解

5.2.3.1 碰撞检测

为减小算力，我们首先采用边界框测试缩小查找范围。在板凳 ABCD 外接矩形 $A'B'C'D'$，使 $A'B'C'D'$ 两边分别平行于 x、y 轴。调用边界框测试算法判断 ${}^iA'^iB'^iC'^iD'$ 与 ${}^jA'^jB'^jC'^jD'$ 间是否相交。若不相交，则 ${}^iA^iB^iC^iD$ 与 ${}^jA^jB^jC^jD$ 一定不相碰。缩小查找范围至 ${}^iA'^iB'^iC'^iD'$ 与 ${}^jA'^jB'^jC'^jD'$ 相交的板凳。

接着根据分离轴定理（SAT）将矩形投影到各个分离轴，检查区域重叠情况，以此判断矩形碰撞情况。

Figure 9: 碰撞检测流程图

5.2.3.2 优化模型的求解

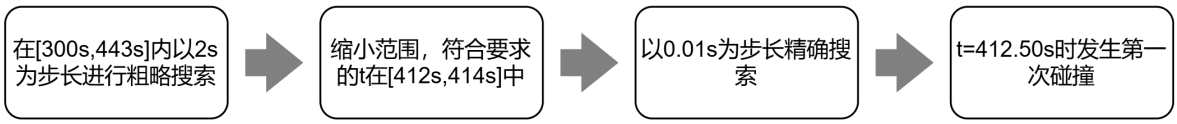

Figure 10: 优化模型求解思路流程图

由于板凳的长度 l 一定，排列在等距螺线上的把手对应极角 θ 越大，相邻板凳间所成角度越大，越不容易发生碰撞。考虑到计算机算力限制，我们只讨论前五十个板凳的碰撞情况。采用双重循环遍历法对任两个板凳进行碰撞检测，但值得注意的是，相邻两个板凳即使通过了碰撞检测也并不会发生碰撞，在编程中要排除这种情况。

我们采用定步长搜索法求解该优化模型。首先，在 $[300s, 443s]$ 内进行步长为 2s 的定步长搜索，在不同时间取值下对板凳进行碰撞判定。初步可判定首次碰撞发生在 $t \in [412s, 414s]$。接着缩小范围，对 $t \in [412s, 414s]$ 以等步长 $\Delta t = 0.01s$ 进行搜索，在不同时间下进行碰撞检测。搜索到在 $t = 412.50s$ 时，第 0 条板凳（龙头）与第 8 条板凳发生碰撞，碰撞时龙头的坐标为 (x,y) 为 (1.2149462307026162，1.93937824161999871)。

将龙头坐标代入螺线参数方程（式 (6)），得到 θ_0，采用牛顿迭代法求解式 (7)、式 (9)，得到各个把手对应极角 θ_i 和速度 v_i。将 θ_i 回代螺线参数方程（式 (6)），得到各把手位置坐标（同问题一）。部分结果如表 4所示，完整结果见文件 result2.xlsx。

Table 4: 各把手位置坐标与速度

	横坐标 x (m)	纵坐标 y (m)	速度 (m/s)
龙头	1.214946231	1.939378242	1
第 1 节龙身	-1.639254701	1.757342425	0.998584234
第 51 节龙身	1.286844527	4.324793243	0.993935162
第 101 节龙身	-0.542120896	-5.879511307	0.993191856
第 151 节龙身	0.96298434	-6.958217815	0.992887362
第 201 节龙身	-7.894004031	-1.224925021	0.992721558
龙尾（后）	0.962069125	8.321999486	0.992670341

5.3 问题三：掉头空间最小螺距的求解

5.3.1 优化模型的建立

5.3.1.1 静态优化模型

设龙头前把手盘入边界圆盘入点为 B。在静态模型中我们不考虑时间 t 的变化，仅以龙头前把手与 B 重合时龙身不发生碰撞为约束条件，最小化螺距 p 为目标构建优化模型。**约束条件的确立** 引入问题二中碰撞模型，碰撞矩形的四个顶点 $(x_1', y_1')\pm\boldsymbol{v}, (x_2', y_2')\pm\boldsymbol{v}$，任两个矩形间都没有重叠则盘龙没有发生碰撞：

$$\Omega_{(i,t=t_0)} \cap C_R\Omega_{(i,t=t_0)} = \emptyset, i = 0, 1, 2, ..., 223 \tag{23}$$

优化模型的确立

$$\begin{aligned} &\min \quad p \\ &s.t. \begin{cases} r = \dfrac{p}{2\pi}\theta + 16p, \theta < 0 \\ \begin{bmatrix} x_{(i,t_0)} \\ y_{(i,t_0)} \end{bmatrix} = r(\theta) \begin{bmatrix} cos\theta \\ sin\theta \end{bmatrix} \\ \Omega_{(i,t=t_0)} \cap C_R\Omega_{(i,t=t_0)} = \emptyset, i = 0, 1, 2, ..., 223 \end{cases} . \end{aligned} \tag{24}$$

5.3.1.2 动态优化模型

实际情况中，队伍可能在进入调头区前发生碰撞。我们在静态模型的基础上加入对时间变量 t 的考量，理论上能建立更精确的约束条件，得到更精确的优化结果。**约束条件的确立** 引入问题二中碰撞模型，任两个矩形间都没有重叠则盘龙没有发生碰撞：

$$\Omega_{(i,t)} \cap C_R\Omega_{(i,t)} = \emptyset, i = 0, 1, 2, ..., 223 \tag{25}$$

优化模型的确立

$$\begin{aligned} &\min \quad p \\ &s.t. \begin{cases} r = \dfrac{p}{2\pi}\theta + 16p, \theta < 0 \\ \begin{bmatrix} x_{(i,t)} \\ y_{(i,t)} \end{bmatrix} = r(\theta) \begin{bmatrix} cos\theta \\ sin\theta \end{bmatrix} \\ \Omega_{(i,t)} \cap C_R\Omega_{(i,t)} = \emptyset, i = 0, 1, 2, ..., 223 \end{cases} . \end{aligned} \tag{26}$$

5.3.2 优化模型的求解

5.3.2.1 静态模型的求解

采用定步长搜索法求解优化模型。此时龙头前把手位置与 B 重合，又螺线中心与极点重合，则 $r(\theta_B) = 4.5m$。将龙头前把手对应极角 θ_B 作为初始值代入式 (7)，采用牛顿迭代法迭代求解得各个把手的位置。

引用问题二中的碰撞检测模型检测对于当前螺距 p 任两个板凳是否会发生碰撞。对 p 以等步长 $\Delta p = 0.001m$ 从 0.55m 向下搜索，当有碰撞发生时，停止搜索。搜索得不会发生碰撞的 $p_{min} = 0.420m$。

5.3.2.2 动态模型的求解

在静态模型求解的基础上，加入以 $\Delta t = 10s$ 遍历时间 $t \in (0, 300s)$ 的循环求解。接着缩小范围至 $t \in [200s, 210s]$，以 $\Delta t = 1s$ 再次遍历，搜索得不会发生碰撞的 $p_{min} = 0.448m$。

5.3.3 模型优化与误差分析

5.3.3.1 模型优化

显然，Δp 越小，误差越小。但 Δp 越小，计算也越复杂，因此 Δp 的选取应该权衡考虑。

Table 5: 不同精度下 p 值及碰撞节数

Δp	p		碰撞节数	
	静态	动态	静态	动态
10^{-2}	0.42	0.44	第 4 节与第 23 节碰	第 0 节与第 17 节碰
10^{-3}	0.420	0.448	第 4 节与第 23 节碰	第 0 节与第 17 节碰
10^{-4}	0.4202	0.4486	第 4 节与第 23 节碰	第 0 节与第 17 节碰
10^{-5}	0.42021	0.44869	第 4 节与第 23 节碰	第 0 节与第 17 节碰
10^{-6}	0.420209	0.448697	第 4 节与第 23 节碰	第 0 节与第 17 节碰

由上表发现 $\Delta p = 10^{-3}$ 时，得到的结果已经很让人满意，且 $\Delta p = 10^{-3}$ 的继续增大不会带来 p 的精确，但计算机计算的复杂度却明显增加，因此取 $\Delta p = 10^{-3}$。

由表 5还可知碰撞点不随着精度变化发生改变，模型自洽。

5.3.3.2 误差分析

下面绘出 $p-\Delta p$ 图像：

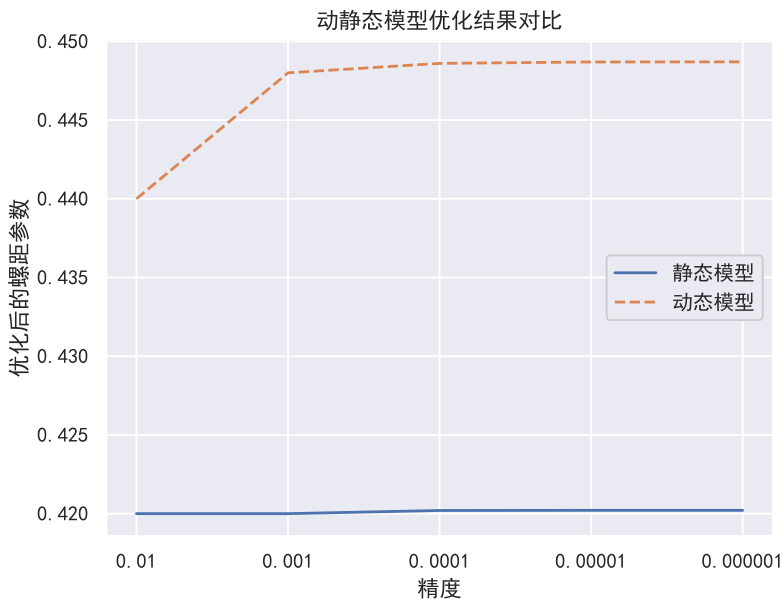

Figure 11: 静态模型与动态模型 $p-\Delta p$ 图像

从图像可知静态模型的收敛性更好，而动态模型在较低精度时 p 偏离较多，可能由于动态模型循环次数较多，算法不够优化导致。但随着精度增加，静态模型与动态模型均表现出较好的收敛性。

两种模型求解答案间有 2cm 左右系统误差。动态模型求解结果螺距更大，龙头和第 17 条龙身相撞可能是因为在动态过程中，龙头前把手到达 B 点前就已经发生第一次相碰。而静态模型中直接将龙头与 B 重叠时作为第一次碰撞不够准确，但这个误差几乎可以忽略（6.35%），即为了减少计算量，静态模型也是可以使用的。

5.4 问题四：调头路径规划

5.4.1 模型的准备

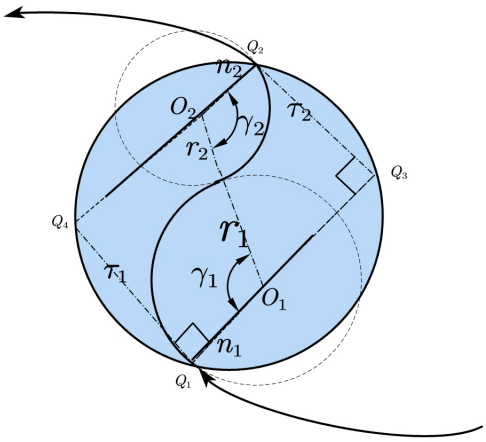

Figure 12: 调头过程主要参数示意图

调头路径由两个相切的圆弧构成,第一个圆弧 C_1 主要控制参数 $(Q_1, O_1, r_1, \tau 1, n_1, \gamma_1)$，第二个圆弧 C_2 主要控制参数 $(Q_2, O_2, r_2, \tau 2, m_2, n_2, \gamma_2)$。

其中 Q_1, Q_2 分别为龙头前把手盘入点和盘出点；r_1, r_2 分别为 C_1, C_2 半径；O_1, O_2 分别为 C_1, C_2 圆心；τ_1, τ_2 分别为盘入、盘出螺线在 Q_1, Q_2 处切线；n_1, n_2 分别为盘入、盘出螺线在 Q_1, Q_2 处法线；m_1, m_2 分别为 Q_1, Q_2 处垂直极径的基准线；γ_1, γ_2 分别为两个圆弧对应圆心角。

5.4.2 模型的建立

5.4.2.1 几何模型的建立

$r_1 + r_2$ 为定值的证明 在本问中盘入螺距和调头空间已知，可写出盘入螺线方程：

$$r = \frac{p}{2\pi}\theta + 16p, \tag{27}$$

其中，本问 p=1.7m。

当 $r(\theta) = 4.5m$ 时，可求出 θ。即盘入点坐标 Q_1 可确定，又盘出螺线与盘入螺线关于螺线中心呈中心对称，则可确定盘出点坐标 Q_2。

调头路径 C_1 与盘入螺线相切，则圆心 O_1 在法线 n_1 上，半径 $r_1 = O_1Q_1$。同理可在 n_2 上找到圆心 O_2 与半径 r_2。又由于圆弧 C_1, C_2 相切，有：

$$O_1O_2 = r_1 + r_2, \tag{28}$$

由矩形的判定可知直线 n_1 与直线 τ_2 交于大圆上一点，将该点令为 Q_3；同理，n_2 与 τ_1 也交于圆上一点 Q_4。

下面在矩形 $Q_1Q_2Q_3Q_4$ 中讨论：矩形长宽均为定值，分别令为 l_2、l_1。过 O_1 作 $\tau_1'//\tau_1$ 交 n_2 于 O_1'。则

$$O_1'O_2 = l_2 - r_1 - r_2 \tag{29}$$

$$O_1O_1' = l_1 \tag{30}$$

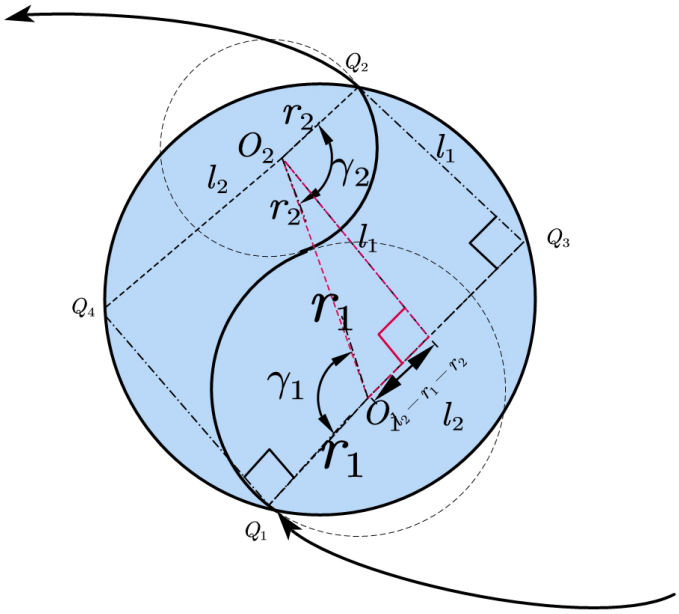

Figure 13: 直角三角形 $O_1O_1'O_2$ 中勾股定理

如图 13，结合式 (28)、式 (29)、式 (30) 在直角三角形 $O_1O_1'O_2$ 中构建勾股定理：

$$l_1^2 + (l_2 - r_1 - r_2)^2 = (r_1 + r_2)^2, \tag{31}$$

化简得：

$$r_1 + r_2 = \frac{l_1^2 + l_2^2}{2l_2}, \tag{32}$$

即 $r_1 + r_2$ 为定值。当 $r_1 = 2r_2$ 时，可以求得 r_1、r_2。

调头曲线长度为定值的证明 由 $n_1//n_2$ 可知 $\gamma_1 = \gamma_2$，令其大小为 γ。

$$\gamma = \angle O_1'O_1O_2 + \frac{\pi}{2} = arccos\frac{2l_1l_2}{l_1^2 + l_2^2} + \frac{\pi}{2}, \tag{33}$$

即 γ 为定值。调头曲线长度：

$$L = r_1\gamma + r_2\gamma = (r_1 + r_2)\gamma, \tag{34}$$

即调头曲线长度仅与 r_1+r_2 有关，而 r_1+r_2 为定值。即不论如何调整圆弧，都不可能使得调头曲线变短。

5.4.2.2 微分方程模型的建立——$\theta^{param}-t$ 关系

此处 θ^{param} 为关于时间 t 的分段函数。当把手处于圆弧上时，线速度 v 一定，则角速度为定值，θ^{param} 与 t 有线性关系：

$$\theta^{param}=kt+m_0, \tag{35}$$

当把手处于螺线上时，同 5.1.1.2 建立微分方程模型。盘入时微分方程模型同式 (5)，下面讨论盘出时情况。板凳龙以逆时针盘出：

$$r=\frac{p}{2\pi}(\pi+\theta^{param}),\theta^{para}>0, \tag{36}$$

因为舞龙队顺时针转动，则 $\theta^{param}>0$。如 5.1.1.2 中分解线速度 v 构建微分方程：

$$v=\sqrt{r(\theta)^2+r'(\theta)^2}\cdot\frac{d\theta}{dt}\theta^{param}|_{t=??}=?? \tag{37}$$

其中初值条件在龙头刚进入盘出螺线时取得。

5.4.2.3 各把手位置参数与速度递推模型的确立

将 $r_1=2r_2$ 代入式 (32)，求得：

$$\begin{aligned} r_1&=\frac{l_1^2+l_2^2}{3l_2}\\ r_2&=\frac{l_1^2+l_2^2}{6l_2}\end{aligned} \tag{38}$$

则板凳龙沿新曲线 S 盘动。新曲线由盘入螺线、半径分别为 r_1、r_2 的圆弧 C_1、C_2、盘出螺线四部分构成，我们分为两种情况建模求解位置参数与速度递推公式。

情况一：螺线部分与交界部分 该部分引用 5.1.1.3 中把手位置与速度求解模型。首先将各曲线方程写为参数方程形式，盘入、盘出螺线参见式 (6)。为使结果更直观，**圆弧 C_1、C_2 参数不再使用 θ：**

$$\begin{cases} x=r_1cos\theta_{param1}+x_1\\ y=r_1sin\theta_{param1}+y_1\end{cases}, \tag{39}$$

$$\begin{cases} x=r_2cos\theta_{param2}+x_2\\ y=r_2sin\theta_{param2}+y_2\end{cases}, \tag{40}$$

其中，(x_1, y_1)、(x_2, y_2) 分别为 O_1、O_2 坐标。θ_{param1} 和 θ_{param2} 为圆弧 C_1、C_2 参数方程的参数，$\theta_{param1} \in [0, 2\pi]$，$\theta_{param2} \in [-\pi, \pi]$。

将式 (6)、(39)、(40) 参数方程代入式 (7)，得到位置参数 θ_i^{param} 和 θ_{i+1}^{param} 递推关系。定义 $\Delta\theta_i^{param}$：

$$\Delta\theta_i^{param} := \theta_{i+1}^{param} - \theta_i^{param}, \tag{41}$$

值得注意的是此处 θi^{param} 不再是极角，而是不同曲线参数方程的参数。对于交界部分，θi^{param} 为前把手所在曲线对应参数，θ_{i+1}^{param} 为后把手所在曲线对应参数。对于螺线部分，θi^{param} 为极角。

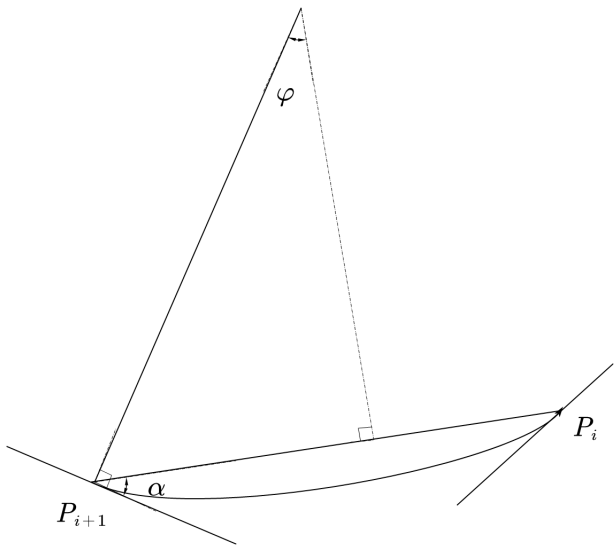

Figure 14: $\alpha = \varphi$

如图 14，对于圆弧 C_2，有 $\alpha = \varphi$。

$$\alpha = \varphi = arccos\frac{d_{O_2 \to P_i P_{i+1}}}{r}, \tag{42}$$

其中，$O_2(x_2, y_2)$ 到直线 $P_i P_{i+1}$ 距离可由点到直线距离公式得到：

$$d_{O_2 \to P_i P_{i+1}} = \frac{|ax_2 + by_2 + c|}{\sqrt{a^2 + b^2}}, \tag{43}$$

同理可求解情况②下 α：

$$\alpha = \varphi = arccos\frac{d_{O_1 \to P_i P_{i+1}}}{r}, \tag{44}$$

$$d_{O_1 \to P_i P_{i+1}} = \frac{|ax_2 + by_2 + c|}{\sqrt{a^2 + b^2}} \tag{45}$$

对于螺线，$\alpha = arctan\frac{1}{\theta}$（式 (11)）。$\beta$ 同式 (13)。将 α、β 代入式 (9)，得到 v_i 与 v_{i+1} 速度递推公式，

情况二：圆弧部分 由于圆弧具有对称性式，当 $P_iP_{i+1}=d$ 时，$\Delta\theta$ 为定值，且前后把手速度相同。此部分递推关系较好推导：

$$\begin{cases} \theta_{i+1}^{param}-\theta_i^{param}=2arcsin\dfrac{d}{2r_{1,2}} \\ v_{i+1}=v_i \end{cases}, \tag{46}$$

将式 (38) 中 r_1、r_2 代入上式即可解得圆弧上 θ_{i+1}^{param} 相对 θ_i^{param} 的递推关系。

5.4.3 模型求解

5.4.3.1 $\theta^{param}-t$ 关系的求解

将圆弧 C_1 和 C_2 参数代入式 (35)，得到分段函数圆弧段：

$$\theta^{param}=\begin{cases} -\dfrac{v}{r_1}t+\phi_0-\alpha, t\in(0,\dfrac{r_1\gamma}{v}) \\ \dfrac{v}{r_2}t+\pi+\phi_0-\gamma-\alpha, t\in(\dfrac{\gamma r_1}{v},\dfrac{\gamma(r_1+r_2)}{v}) \end{cases}, \tag{47}$$

两个方程中的常数项作用均为调整 θ^{param} 至取值范围。γ 为圆弧对应圆心角，α 为 P_i 处曲线切线 τ 与基准线 m 所成角。ϕ_0 为 Q_1 的极角。

参照 5.1.2.2 求解微分方程，盘入微分方程特解同式 (17)，盘出微分方程特解为：

$$\frac{1}{2}(r\sqrt{r^2+k^2}+k^2ln(r+\sqrt{r^2+k^2}))=kv\cdot t+\frac{1}{2}k^2lnk, t\in(\frac{\gamma(r_1+r_2)}{v},100] \tag{48}$$

其中，$k=\dfrac{p}{2\pi}$，r 满足分段函数关系。则 $\theta^{param}-t$ 函数关系：

$$f:\begin{cases} \frac{1}{2}(r\sqrt{r^2+k^2}+k^2ln(r+\sqrt{r^2+k^2}))=-kv\cdot(t-300), t\in[-100,0] \\ \theta^{param}=-\dfrac{v}{r_1}t+\phi_0-\alpha, t\in(0,\dfrac{r_1\gamma}{v}] \\ \theta^{param}=\dfrac{v}{r_2}t+\pi+\phi_0-\gamma-\alpha, t\in(\dfrac{\gamma r_1}{v},\dfrac{\gamma(r_1+r_2)}{v}] \\ \frac{1}{2}(r\sqrt{r^2+k^2}+k^2ln(r+\sqrt{r^2+k^2}))=kv\cdot t+\frac{1}{2}k^2lnk, t\in(\dfrac{\gamma(r_1+r_2)}{v},100] \end{cases} \tag{49}$$

注意此处 θ^{param} 并不是极角，具体见 5.4.2.3 情况一。

5.4.3.2 各把手位置和速度的求解——迭代法

对于递推关系初始值 $\theta_0=\theta^{param}$，θ^{param} 即为龙头前把手在 t 时刻位置所对应的位置参数，满足式 (49)。

采用牛顿迭代法求解 $\Delta\theta_i^{param}$，将 $\Delta\theta_i^{param}$ 代入式 (41)，可求出每个把手对应 θ_i^{param}。将 θ_i^{param} 代入路径曲线参数方程 (49) 得到时间 t 时各个把手的位置。以等步长 $\Delta t=1s$

在 [-100s,100s] 范围内离散时间变量 t，得到部分结果如表 6所示，完整结果见文件 result4.xlsx。

Table 6: 把手位置（部分）

	0s	60s	120s	180s	240s	300s
龙头 x (m)	8.80	5.80	-4.08	-2.96	2.59	4.42
龙头 y (m)	-0.00	-5.77	-6.30	6.09	-5.36	2.32
第 1 节龙身 x (m)	8.80	7.46	-1.45	-5.24	4.82	2.46
第 1 节龙身 y (m)	2.86	-3.44	-7.41	4.36	-3.56	4.40
第 51 节龙身 x (m)	8.80	8.80	-5.54	2.89	6.00	-6.30
第 51 节龙身 y (m)	85.36	25.46	6.38	7.25	-3.83	0.47
第 101 节龙身 x (m)	8.80	8.80	8.80	1.90	-4.92	-6.24
第 101 节龙身 y (m)	167.86	107.96	48.08	-8.47	-6.38	3.94
第 151 节龙身 x (m)	8.80	8.80	8.80	8.80	8.80	7.04
第 151 节龙身 y (m)	250.36	190.46	130.58	70.71	10.89	4.39
第 201 节龙身 x (m)	8.80	8.80	8.80	8.80	8.80	8.80
第 201 节龙身 y (m)	332.86	272.96	213.08	153.21	93.39	33.63
龙尾（后）x (m)	8.80	8.80	8.80	8.80	8.80	8.80
龙尾（后）y (m)	367.51	307.61	247.73	187.86	128.04	68.28

同理，将初始值龙头前把手速度 $v_0 = 1m/s$ 代入递推式（见 5.4.2.3），运用牛顿迭代法解得各个把手的速度。部分结果如表 7所示，完整结果见文件 result4.xlsx。

Table 7: 把手速度（部分）

	0s	60s	120s	180s	240s	300s
龙头 (m/s)	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000
第 1 节龙身 (m/s)	0.99995	0.99999	0.99999	0.99998	0.99997	0.99995
第 51 节龙身 (m/s)	0.99995	0.99732	0.99986	0.99980	0.99968	0.99943
第 101 节龙身 (m/s)	0.99995	0.99732	0.99852	0.99968	0.99952	0.99918
第 151 节龙身 (m/s)	0.99995	0.99732	0.99852	0.99912	0.99743	0.99904
第 201 节龙身 (m/s)	0.99995	0.99732	0.99852	0.99912	0.99743	0.99634
龙尾（后）(m/s)	0.99995	0.99732	0.99852	0.99912	0.99743	0.99634

5.5 问题五：最大行进速度的确定

5.5.1 模型的准备

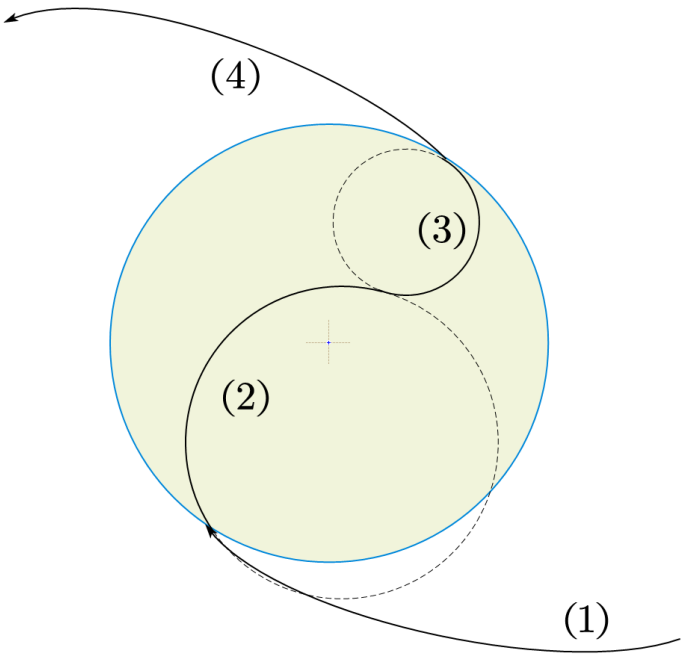

Figure 15: 区间的划分

如图 15，舞龙队伍行进路线被分为四个区域。区域①为盘入螺线，区域②为调头圆弧 C_1，区域③为调头圆弧 C_2，区域④为盘出螺线。

5.5.1.1 螺线上沿极径增大方向把手速度递减的证明

由表 result1.xlsx 可知，当 $t \in (0, 300s)$ 时，对任意时刻有 $v_{i+1} > v_i$，即对于在螺线上运动的把手沿极径增大的方向速度递减。

5.5.2 模型的建立

5.5.2.1 队伍最大速度初步锁定

盘龙运动情况共以下四种：

1. **龙头位于区域 ① 时：**由于在螺线上沿极径增大方向把手速度递减规律，则此时龙身速度均比龙头小，即此时舞龙队各把手速度最大值为 v。
2. **龙头位于区域 ② 时：**由式（46）可知位于 ② 上的龙身均以和龙头相同的速度前进。
3. **龙头位于区域 ③ 时：**同上，位于 ③ 上的龙身均以和龙头相同的速度前进。
4. **龙头位于区域 ④ 时：**由于螺线上沿极径增大方向把手速度递减规律，位于 ④ 上的龙身越靠近 Q_2，速度越大，即此时舞龙队速度最大值可能在刚进入盘出螺线的把手处取得。

综上，舞龙队伍最大速度只会在距离盘出螺线最近的把手 P_n 的速度 v_n、距离盘入螺线最近的把手 P_f 的速度 v_f、圆弧上匀速速度 v_{circ1}、v_{circ2} 中产生。

5.5.2.2 优化模型

建立以龙头行进速度最大为目标，舞龙队各把手的速度均不超过 2m/s 为约束条件的优化模型：

$$\begin{aligned} \min \quad & v_0 \\ s.t. \quad & v_n, v_f, v_{circ1}, v_{circ2} \leq 2m/s. \end{aligned} \tag{50}$$

5.5.3 模型的求解

以步长对 $t \in$ 进行遍历，初步确定舞龙队伍在 $t \in$ 内某时刻达到速度最大，接着以步长对 $t \in$ 进行进一步遍历，搜索得到 t= 时，第节龙身达到速度最大值 2m/s。将 v 代入问题四速度递推公式，此时龙头速度为 1.58m/s。

参考文献

无参考文献

第一问代码

```
from math import pi,sqrt,cos,log,sin,atan,tan
import numpy as np
from scipy.optimize import fsolve

def head_position_per_second():
    #求解每一时刻龙头位置
    k=0.55/(2*pi)
    C=0.5*k**2*log(k)
    def theta(r):#定义两个转换工具
        return r/k
    def r_(theta_):
        return k*theta_
    rA=r_(32*pi)    #初始时刻 r
    tA=(0.5*(rA*sqrt(rA**2+k**2)+k**2*log(rA+sqrt(rA**2+k**2)))-C)/k#求解顺序时刻 A 点对应
    t=[tA-i for i in range(0,301)]
    rr=[]
    x=[]
    y=[]
    theta_=[]
    for i in t:
        #求解龙头的每一个时刻的位置
        def equation(r,t=i):
            return 0.5*(r*sqrt(r**2+k**2)+k**2*log(r+sqrt(r**2+k**2)))-k*t-C

        # 使用 fsolve 查找方程的根
        initial_guess = 1    # 初始猜测值
        solution = fsolve(equation, initial_guess)
        r=solution[0]#极径
        rr.append(r)
        a=theta(r)#每个时刻的对应极角
        theta_.append(a)
        x0=r*cos(a)
        y0=r*sin(a)
        x.append(x0)
        y.append(y0)
    return x,y

class constant(object):
    def __init__(self):
        self.num_chair=223
        self.d_head=0.01*(341-2*27.5)
        self.d_gen=0.01*(220-2*27.5)
```

```
constant=constant()

def xy_to_polar(x,y):
    r0=0.55
    r=np.sqrt(x**2+y**2)
    theta=2*pi*(r/r0)
    return r,theta

#已知每个时刻的龙头位置 x,y(x_head,y_head)
#迭代二分法找对应点
def point_equation(delta_theta,x1,y1,r0,d):
    #选取点与上一位置点的距离差
    _,theta1=xy_to_polar(x1,y1)
    theta2=theta1+delta_theta
    x2=r0*theta2*cos(theta2)/(2*pi)
    y2=r0*theta2*sin(theta2)/(2*pi)
    distance=(x1-x2)**2+(y1-y2)**2
    return distance-d**2

def count_position(x_head,y_head,constant):
    #得出每一点的位置(x,y,r)
    xs,ys,rs,r0=[x_head],[y_head],[sqrt(x_head**2+y_head**2)],0.55
    positions_moment=[x_head,y_head]
    for i in range(constant.num_chair):
        x,y=xs[i],ys[i]
        if i==0:
            delta_theta=fsolve(point_equation,
pi/4,args=(x,y,r0,constant.d_head))
        else:
            delta_theta=fsolve(point_equation, pi/4,args=(x,y,r0,constant.d_gen))
        assert delta_theta>0, "Value must be positive"
        _,theta1=xy_to_polar(x,y)
        theta2=theta1+delta_theta
        r_new=theta2*r0/(2*pi)
        if r_new>16*0.55:
            break
        x_new=r0*theta2*cos(theta2)/(2*pi)
        y_new=r0*theta2*sin(theta2)/(2*pi)
        xs.append(x_new)
        ys.append(y_new)
        positions_moment.append(x_new)
        positions_moment.append(y_new)
        rs.append(r_new)
```

```
    while len(ys)<constant.num_chair:
        i=1
        if i==1:
            if len(ys)==1:
                y_new=sqrt(constant.d_head**2-(16*0.55-xs[-1])**2)+ys[-1]
            else:
                y_new=sqrt(constant.d_gen**2-(16*0.55-xs[-1])**2)+ys[-1]
        else:
            y_new=ys[-1]+constant.d_gen
        xs.append(16*0.55)
        ys.append(y_new)
        positions_moment.append(16*0.55)
        positions_moment.append(y_new)
        rs.append(sqrt((16*0.55)**2+y_new**2))
        i+=1
    return xs,ys,positions_moment,rs

def count_angle(xs,ys,rs,constant):
    #计算每条凳子前端与后端与相应切线或者竖直直线的夹角
    def linear_para(x1,y1,x2,y2):#得出一般直线方程系数
        return y1-y2,x2-x1
    angles_front,angles_behind,r0=[],[],0.55
    for i in range(constant.num_chair):
        if xs[i]==16*0.55:
            angles_front.append(0)
            angles_behind.append(0)
        elif xs[i]!=16*0.55 and xs[i+1]==16*0.55:
            alpha1=atan(r0/(2*pi*rs[i]))
            A,B=linear_para(xs[i],ys[i],xs[i+1],ys[i+1])
            beta=atan((A*tan(2*pi*rs[i]/r0)-B)/(A+B*tan(2*pi*rs[i]/r0)))
            angles_front.append(alpha1-beta)
            angles_behind.append(atan(B/A))
        else:
            alpha1=atan(r0/(2*pi*rs[i]))
            alpha2=atan(r0/(2*pi*rs[i+1]))
            A,B=linear_para(xs[i],ys[i],xs[i+1],ys[i+1])
            beta=atan((A*tan(2*pi*rs[i]/r0)-B)/(A+B*tan(2*pi*rs[i]/r0)))
            angles_front.append(abs(alpha1-beta))
            angles_behind.append(abs(alpha2-beta))
    return angles_front,angles_behind

def count_velocity(angles_front,angles_behind):
    velocitys=[1]
    for i in range(constant.num_chair):
```

```
            velocity_new=velocitys[i]*cos(angles_front[i])/cos(angles_behind[i])
            velocitys.append(velocity_new)
        return velocitys

time=300
xs_head,ys_head=head_position_per_second()
positions,velocitys=[],[]
for i in range(time+1):
        x_head,y_head=xs_head[i],ys_head[i]
        xs,ys,positions_moment,rs=count_position(x_head, y_head,constant)
        positions.append(positions_moment)
        angles_front,angles_behind=count_angle(xs, ys, rs, constant)
        velocitys_moment=count_velocity(angles_front,angles_behind)
        velocitys.append(velocitys_moment)


def ensure_numeric(lst):
        return [[float(x) if not np.isnan(x) else np.nan for x in sublist] for sublist in lst]
positions = ensure_numeric(positions)
velocitys = ensure_numeric(velocitys)

'''
from openpyxl import load_workbook
positions = np.array(positions)
velocitys = np.array(velocitys)
positions = positions.T
velocitys = velocitys.T
file_path = "C:/Users/AlbertYang/Desktop/2024 国赛/A 题/附件/result1.xlsx"
wb = load_workbook(file_path)
start_row = 2
start_col = 2
ws = wb.active
ws1 = wb['位置']
ws2 = wb['速度']
for i, row in enumerate(positions):
        for j, value in enumerate(row):
            ws1.cell(row=start_row + i, column=start_col + j, value=value)
for i, row in enumerate(velocitys):
        for j, value in enumerate(row):
            ws2.cell(row=start_row + i, column=start_col + j, value=value)
wb.save(file_path)
'''
```

第二问代码

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
sns.set()
from math import pi, log, sqrt, cos, sin
import numpy as np
from scipy.optimize import fsolve

k = 0.55 / (2 * pi)
C = 0.5 * k**2 * log(k)

def theta(r):
    return r / k

def r_(theta_):
    return k * theta_

#%%求各个时刻龙头位置
rA = r_(32 * pi)
tA = (0.5 * (rA * sqrt(rA**2 + k**2) + k**2 * log(rA + sqrt(rA**2 + k**2))) - C) / k
t = [tA - i for i in range(0, 301)]
rr = []
x = []
y = []
theta_ = []

for i in t:
    def equation(r, t=i):
        return 0.5 * (r * sqrt(r**2 + k**2) + k**2 * log(r + sqrt(r**2 + k**2))) - k * t
- C

    initial_guess = 1
    solution = fsolve(equation, initial_guess)
    r = solution[0]
    rr.append(r)
    a = theta(r)
    theta_.append(a)
    x0 = r * cos(a)
    y0 = r * sin(a)
    x.append(x0)
    y.append(y0)

# 画散点图
plt.plot(x, y, color='blue')
# 显示图形
plt.show()
```

```
#%%求龙身上点位置
class constant(object):
    def __init__(self):
        self.num_chair = 223
        self.d_head = 0.01 * (341 - 2 * 27.5)
        self.d_gen = 0.01 * (220 - 2 * 27.5)

constant = constant()

def point_equation(delta_theta,x1,y1,r0,d):
    #选取点与上一位置点的距离差
    _,theta1=xy_to_polar(x1,y1)
    theta2=theta1+delta_theta
    x2=r0*theta2*cos(theta2)/(2*pi)
    y2=r0*theta2*sin(theta2)/(2*pi)
    distance=(x1-x2)**2+(y1-y2)**2
    return distance-d**2
def xy_to_polar(x, y):
    r0 = 0.55
    r = np.sqrt(x**2 + y**2)
    theta = 2 * pi * (r / r0)
    return r, theta

def count_position(x_head, y_head, constant):
    xs, ys, rs, r0 = [x_head], [y_head], [sqrt(x_head**2 + y_head**2)], 0.55
    for i in range(constant.num_chair):
        x, y = xs[i], ys[i]
        if i == 0:
            delta_theta = fsolve(point_equation, pi / 4, args=(x, y, r0,
constant.d_head))
        else:
            delta_theta = fsolve(point_equation, pi / 4, args=(x, y, r0,
constant.d_gen))
        delta_theta = delta_theta[0]   # 确保是标量
        _, theta1 = xy_to_polar(x, y)
        theta2 = theta1 + delta_theta
        x_new = r0 * theta2 * cos(theta2) / (2 * pi)
        y_new = r0 * theta2 * sin(theta2) / (2 * pi)
        xs.append(x_new)
        ys.append(y_new)
        rs.append(theta2 * r0 / (2 * pi))
    return xs, ys, rs

x_now = x[-1]#300s 时龙图
```

```
y_now = y[-1]
xs, ys, rs = count_position(x_now, y_now, constant)

plt.plot(xs,ys,color='red')
plt.show()
#%%碰撞模型

import geopandas as gpd
from shapely.geometry import Point, Polygon

def compute_rectangle_vertices(x1_, y1_, x2_, y2_,i ,width=0.3):#计算四个角点考虑了延伸和宽度
    if i ==0:
        k=(341-27.5)/(341-55)
    else:
        k=(220-27.5)/(220-55)
    x1=x2_-k*(x2_-x1_)
    y1=y2_-k*(y2_-y1_)
    x2=x1_+k*(x2_-x1_)
    y2=y1_+k*(y2_-y1_)
    d = np.array([x2 - x1, y2 - y1])
    v = np.array([-d[1], d[0]])
    v_norm = v / np.linalg.norm(v)

    offset = (width / 2) * v_norm
    vertices = [
        np.array([x1, y1]) + offset,
        np.array([x2, y2]) + offset,
        np.array([x2, y2]) - offset,
        np.array([x1, y1]) - offset
    ]
    return np.array(vertices)

# 构建矩形对象
def juxin(p,i):
    p0,pf=p
    x1,y1=p0
    x2,y2=pf
    p1=compute_rectangle_vertices(x1, y1, x2, y2,i)[0]
    p2=compute_rectangle_vertices(x1, y1, x2, y2,i)[1]
    p3=compute_rectangle_vertices(x1, y1, x2, y2,i)[2]
    p4=compute_rectangle_vertices(x1, y1, x2, y2,i)[3]
    polygon = Polygon([p1,p2,p3,p4])
    return polygon
```

```
def p_(t,i):#i 是第 i 节龙身的前把手,找把手坐标
    rA = r_(32 * pi)
    tA = (0.5 * (rA * sqrt(rA**2 + k**2) + k**2 * log(rA + sqrt(rA**2 + k**2))) - C) / k
    T=tA-t
    def equation(r, t=T):
        return 0.5 * (r * sqrt(r**2 + k**2) + k**2 * log(r + sqrt(r**2 + k**2))) - k * t
- C

    initial_guess = 1
    solution = fsolve(equation, initial_guess)
    r = solution[0]
    a = theta(r)
    x0 = r * cos(a)
    y0 = r * sin(a)
    xs, ys, rs = count_position(x0, y0, constant)#生成龙身上的节点位置

    return [xs[i],ys[i]],[xs[i+1],ys[i+1]]

def decide(t):
    for i in range(0,51):
        for j in range(2,51):#一定要跳过 i+1，相邻的一定是重叠的
            i_=i+j
            ju1=juxin(p_(t,i),i)
            ju2=juxin(p_(t,i_),i_)
            if ju1.intersects(ju2):#intersects 函数内置边界框测试与分离轴定理
判断矩形是否相交
                return 'T',i,i_
        return 'F'

dt=0.01
tt=[]
for t in np.arange(412,414,dt):
    if decide(t)!='F':
        print('在',t,'秒撞上')
        a,b,c=decide(t)
        print('第{',b,'}节与第{',c,'}节撞上了')
        tt.append(t)
        break
print('碰撞时的龙头坐标(x,y)：',p_(tt[0],0)[0])
```

第三问代码

```
#静态模型

import seaborn as sns
```

```
import matplotlib.pyplot as plt
sns.set()
from math import pi, log, sqrt, cos, sin
import numpy as np
from scipy.optimize import fsolve
from shapely.geometry import Polygon
import geopandas as gpd

class Constant:
    def __init__(self):
        self.num_chair = 223
        self.d_head = 0.01 * (341 - 2 * 27.5)
        self.d_gen = 0.01 * (220 - 2 * 27.5)

constant = Constant()

def theta(r, p):
    k = p / (2 * pi)
    C = 0.5 * k**2 * log(k)
    return r / k

def r_(theta_, p):
    k = p / (2 * pi)
    C = 0.5 * k**2 * log(k)
    return k * theta_

def point_equation(delta_theta, x1, y1, p, d):
    r0 = p
    _, theta1 = xy_to_polar(x1, y1, p)
    theta2 = theta1 + delta_theta
    x2 = r0 * theta2 * cos(theta2) / (2 * pi)
    y2 = r0 * theta2 * sin(theta2) / (2 * pi)
    distance = (x1 - x2)**2 + (y1 - y2)**2
    return distance - d**2

def xy_to_polar(x, y, p):
    r0 = p
    r = np.sqrt(x**2 + y**2)
    theta = 2 * pi * (r / r0)
    return r, theta

def count_position(x_head, y_head, constant, p):
    xs, ys, rs, r0 = [x_head], [y_head], [sqrt(x_head**2 + y_head**2)], p
    for i in range(constant.num_chair):
```

```
            x, y = xs[i], ys[i]
            if i == 0:
                delta_theta = fsolve(point_equation, pi / 4, args=(x, y, r0, constant.d_head), xtol=1e-10)[0]
            else:
                delta_theta = fsolve(point_equation, pi / 4, args=(x, y, r0, constant.d_gen), xtol=1e-10)[0]
            _, theta1 = xy_to_polar(x, y, p)
            theta2 = theta1 + delta_theta
            x_new = r0 * theta2 * cos(theta2) / (2 * pi)
            y_new = r0 * theta2 * sin(theta2) / (2 * pi)
            xs.append(x_new)
            ys.append(y_new)
            rs.append(theta2 * r0 / (2 * pi))
    return xs, ys, rs

def compute_rectangle_vertices(x1_, y1_, x2_, y2_, i, width=0.3):
    if i == 0:
        k = (341 - 27.5) / (341 - 55)
    else:
        k = (220 - 27.5) / (220 - 55)
    x1 = x2_ - k * (x2_ - x1_)
    y1 = y2_ - k * (y2_ - y1_)
    x2 = x1_ + k * (x2_ - x1_)
    y2 = y1_ + k * (y2_ - y1_)
    d = np.array([x2 - x1, y2 - y1])
    v = np.array([-d[1], d[0]])
    v_norm = v / np.linalg.norm(v)

    offset = (width / 2) * v_norm
    vertices = [
        np.array([x1, y1]) + offset,
        np.array([x2, y2]) + offset,
        np.array([x2, y2]) - offset,
        np.array([x1, y1]) - offset
    ]
    return np.array(vertices)

def juxin(p, i):
    p0, pf = p
    x1, y1 = p0
    x2, y2 = pf
    vertices = compute_rectangle_vertices(x1, y1, x2, y2, i)
    polygon = Polygon(vertices)
```

```
    return polygon

def p_(t, i, p):
    rA = r_(32 * pi, p)
    k = p / (2 * pi)
    C = 0.5 * k**2 * log(k)
    tA = (0.5 * (rA * sqrt(rA**2 + k**2) + k**2 * log(rA + sqrt(rA**2 + k**2))) - C) / k
    T = tA - t

    def equation(r, t=T):
        return 0.5 * (r * sqrt(r**2 + k**2) + k**2 * log(r + sqrt(r**2 + k**2))) - k * t
- C

    solution = fsolve(equation, 1, xtol=1e-10)
    r = solution[0]
    a = theta(r, p)
    x0 = r * cos(a)
    y0 = r * sin(a)
    xs, ys, rs = count_position(x0, y0, constant, p)

    return [xs[i], ys[i]], [xs[i + 1], ys[i + 1]]

def decide(t, p):
    for i in range(0, 10):
        for j in range(2, 28):
            i_ = i + j
            ju1 = juxin(p_(t, i, p), i)
            ju2 = juxin(p_(t, i_, p), i_)
            if ju1.intersects(ju2):
                return 'T', i, i_
    return 'F'
#%%
print('静态模型如下')
p = 0.42021
for _ in range(55):
    a_ = theta(4.5, p)
    xx = 4.5 * cos(a_)
    yy = 4.5 * sin(a_)
    xs, ys, rs = count_position(xx, yy, constant, p)

    rB = 4.5
    rA = r_(32 * pi, p)
    k = p / (2 * pi)
    C = 0.5 * k**2 * log(k)
```

```
    tA = (0.5 * (rA * sqrt(rA**2 + k**2) + k**2 * log(rA + sqrt(rA**2 + k**2))) - C) / k
    tB = (0.5 * (rB * sqrt(rB**2 + k**2) + k**2 * log(rB + sqrt(rB**2 + k**2))) - C) / k

    result = decide(tA - tB, p)
    if result != 'F':
        T, i, i_ = result
        print(f'在 p 等于 {p} 时，第 {i} 节与第 {i_} 节发生碰撞')

    p -= 0.000001
    print(p)
#动态模型
import seaborn as sns
import matplotlib.pyplot as plt
sns.set()
from math import pi, log, sqrt, cos, sin
import numpy as np
from scipy.optimize import fsolve
from shapely.geometry import Polygon
import geopandas as gpd

class Constant:
    def __init__(self):
        self.num_chair = 223
        self.d_head = 0.01 * (341 - 2 * 27.5)
        self.d_gen = 0.01 * (220 - 2 * 27.5)

constant = Constant()

def theta(r, p):
    k = p / (2 * pi)
    C = 0.5 * k**2 * log(k)
    return r / k

def r_(theta_, p):
    k = p / (2 * pi)
    C = 0.5 * k**2 * log(k)
    return k * theta_

def point_equation(delta_theta, x1, y1, p, d):
    r0 = p
    _, theta1 = xy_to_polar(x1, y1, p)
    theta2 = theta1 + delta_theta
    x2 = r0 * theta2 * cos(theta2) / (2 * pi)
    y2 = r0 * theta2 * sin(theta2) / (2 * pi)
```

```
    distance = (x1 - x2)**2 + (y1 - y2)**2
    return distance - d**2

def xy_to_polar(x, y, p):
    r0 = p
    r = np.sqrt(x**2 + y**2)
    theta = 2 * pi * (r / r0)
    return r, theta

def count_position(x_head, y_head, constant, p):
    xs, ys, rs, r0 = [x_head], [y_head], [sqrt(x_head**2 + y_head**2)], p
    for i in range(constant.num_chair):
        x, y = xs[i], ys[i]
        if i == 0:
            delta_theta = fsolve(point_equation, pi / 4, args=(x, y, r0,
constant.d_head), xtol=1e-10)[0]
        else:
            delta_theta = fsolve(point_equation, pi / 4, args=(x, y, r0,
constant.d_gen), xtol=1e-10)[0]
        _, theta1 = xy_to_polar(x, y, p)
        theta2 = theta1 + delta_theta
        x_new = r0 * theta2 * cos(theta2) / (2 * pi)
        y_new = r0 * theta2 * sin(theta2) / (2 * pi)
        xs.append(x_new)
        ys.append(y_new)
        rs.append(theta2 * r0 / (2 * pi))
    return xs, ys, rs

def compute_rectangle_vertices(x1_, y1_, x2_, y2_, i, width=0.3):
    if i == 0:
        k = (341 - 27.5) / (341 - 55)
    else:
        k = (220 - 27.5) / (220 - 55)
    x1 = x2_ - k * (x2_ - x1_)
    y1 = y2_ - k * (y2_ - y1_)
    x2 = x1_ + k * (x2_ - x1_)
    y2 = y1_ + k * (y2_ - y1_)
    d = np.array([x2 - x1, y2 - y1])
    v = np.array([-d[1], d[0]])
    v_norm = v / np.linalg.norm(v)

    offset = (width / 2) * v_norm
    vertices = [
        np.array([x1, y1]) + offset,
```

```
            np.array([x2, y2]) + offset,
            np.array([x2, y2]) - offset,
            np.array([x1, y1]) - offset
        ]
        return np.array(vertices)

def juxin(p, i):
        p0, pf = p
        x1, y1 = p0
        x2, y2 = pf
        vertices = compute_rectangle_vertices(x1, y1, x2, y2, i)
        polygon = Polygon(vertices)
        return polygon

def p_(t, i, p):
        rA = r_(32 * pi, p)
        k = p / (2 * pi)
        C = 0.5 * k**2 * log(k)
        tA = (0.5 * (rA * sqrt(rA**2 + k**2) + k**2 * log(rA + sqrt(rA**2 + k**2))) - C) / k
        T = tA - t

        def equation(r, t=T):
            return 0.5 * (r * sqrt(r**2 + k**2) + k**2 * log(r + sqrt(r**2 + k**2))) - k * t
- C

        solution = fsolve(equation, 1, xtol=1e-10)
        r = solution[0]
        a = theta(r, p)
        x0 = r * cos(a)
        y0 = r * sin(a)
        xs, ys, rs = count_position(x0, y0, constant, p)

        return [xs[i], ys[i]], [xs[i + 1], ys[i + 1]]

def decide(t, p):
        for i in range(0, 10):
            for j in range(2, 28):
                i_ = i + j
                ju1 = juxin(p_(t, i, p), i)
                ju2 = juxin(p_(t, i_, p), i_)
                if ju1.intersects(ju2):
                    return 'T', i, i_
        return 'F'
#%%动态模型如下
```

```
p = 0.448699#!!!
for _ in range(55):
	k = p / (2 * pi)
	C = 0.5 * k**2 * log(k)
	rB = 4.5
	tB = (0.5 * (rB * sqrt(rB**2 + k**2) + k**2 * log(rB + sqrt(rB**2 + k**2))) - C) /k

	rA = r_(32 * pi, p)
	tA = (0.5 * (rA * sqrt(rA**2 + k**2) + k**2 * log(rA + sqrt(rA**2 + k**2))) - C) / k
	tf=tA-tB
	print('运动到指定地点要花费时间（s）：',tf)
	dt=10#!!!
	t_=[i for i in np.arange(190,tf,dt)]

	tt=[]
	for t in t_:
		if decide(t,p)!='F':
			print('在',t,'秒撞上')
			a,b,c=decide(t,p)
			tt.append(t)
			print('螺距为：',p,'时','第{',b,'}节与第{',c,'}节撞上了','碰撞时的龙头坐标(x,y)：',p_(tt[0],0,p)[0])
			continue
		#print(t,p)

	p -= 0.000001#!!!
	print('p 现在迭代减小')
```

第四问代码

```
from math import pi,sqrt,cos,sin,atan,tan,asin,acos
from scipy.optimize import fsolve
import numpy as np
from decimal import Decimal
from prob1 import constant
from prob4_head_positions2 import x_head_position2,y_head_position2
#from prob4_calc_r_boundary import r_boundary

def cot(angle):
	return 1/tan(angle)

class figurable_constant(object):
	#所有初始常量以及通过手工数学计算能算出的恒量
	def __init__(self,constant):
		self.d_head=constant.d_head
		self.d_gen=constant.d_gen
```

```
        self.num_chair=constant.num_chair
        self.r_head_boundary=4.67131999
        self.r_gen_boundary=4.67131999
        self.R=4.5
        self.r0=1.7
        self.v=1
        self.theta0=2*pi*self.R/self.r0-4*pi
        self.alpha=atan(self.R/(2*pi*self.r0))
        self.x_in=self.R*cos(self.theta0)
        self.y_in=self.R*sin(self.theta0)

self.d_l1=abs(self.y_in-self.x_in*cot(self.theta0-self.alpha))/sqrt(1+cot(self.alpha-self.t
heta0)**2)

self.d_l2=abs(self.y_in-self.x_in*tan(self.theta0-pi-self.alpha))/sqrt(1+tan(self.alpha-se
lf.theta0)**2)
        self.l1=2*self.d_l2
        self.l2=2*self.d_l1
        self.r1=(self.l1**2+self.l2**2)/(3*self.l2)
        self.r2=self.r1/2
        self.gamma=pi-asin(2*self.l1*self.l2/(self.l1**2+self.l2**2))
        self.gamma1_head=2*asin(self.d_head/(2*self.r1))
        self.gamma2_head=2*asin(self.d_head/(2*self.r2))
        self.gamma1_gen=2*asin(self.d_gen/(2*self.r1))
        self.gamma2_gen=2*asin(self.d_gen/(2*self.r2))
        self.t1_head=self.gamma1_head*self.r1/self.v
        self.t2_head=self.gamma2_head*self.r2/self.v
        self.t1_gen=self.gamma1_gen*self.r1/self.v
        self.t2_gen=self.gamma2_gen*self.r2/self.v
        self.t1_l=self.gamma*self.r1/self.v
        self.t2_l=self.gamma*self.r2/self.v
        self.x_Oin=self.x_in+self.r1/sqrt(1+tan(self.theta0-self.alpha)**2)

self.y_Oin=self.y_in+self.r1*tan(self.theta0-pi-self.alpha)/sqrt(1+tan(self.theta0-pi-self
.alpha)**2)
        self.x_Oout=self.x_in-self.r1/sqrt(1+tan(self.theta0-self.alpha)**2)

self.y_Oout=self.y_in-self.r1*tan(self.theta0-pi-self.alpha)/sqrt(1+tan(self.theta0-pi-se
lf.alpha)**2)

class circle_equation(object):
    #圆的参数方程对象
    def __init__(self,phi0,radius,velocity,x_O,y_O,t_center):
        self.phi0=phi0
```

```
        self.radius=radius
        self.velocity=velocity
        self.x_O=x_O
        self.y_O=y_O
        self.t_center=t_center

    def count_position_by_time(self,t):
        if t<=self.t_center:
            pass
        else:
            t=t-self.t_center
        x=self.radius*cos(self.radius*t/self.velocity+self.phi0)+self.x_O
        y=self.radius*sin(self.radius*t/self.velocity+self.phi0)+self.y_O
        return x,y

    def count_position_by_phase(self,phase):
        x=self.radius*cos(phase)+self.x_O
        y=self.radius*sin(phase)+self.y_O
        return x,y

class helice(object):
    def __init__(self,r0,phi0):
        self.r0=r0
        self.phi0=phi0

    def count_position_by_phase(self,phase):
        x=self.r0*(phase+self.phi0)*cos(phase)/(2*pi)
        y=self.r0*(phase+self.phi0)*sin(phase)/(2*pi)
        return x,y

#%%judge boundary condition
def judge_boundary(xi ,yi, constant, ishead, **kwargs):
    #theta_par_in_b3=constant.theta0-constant.alpha
    theta_par_in_b2=-constant.alpha+(pi-constant.theta0)+(pi-constant.gamma)
    theta_par_out_b2=constant.theta0-pi-constant.gamma-constant.alpha
    #theta_par_out_b1=constant.theta0-constant.alpha+constant.gamma
    r=sqrt(xi**2+yi**2)
    if ishead==1:
        if kwargs['t']==0:
            return 3 ,2
        elif 0<kwargs['t'] and kwargs['t']<=constant.t1_l:
            if kwargs['t']<constant.t1_head:
                return 3 ,1
            return 2 ,2
```

```
        elif constant.t1_l<kwargs['t'] and kwargs['t']-constant.t1_l<=constant.t2_l:
            if kwargs['t']-constant.t1_l<constant.t2_head:
                return 2 ,1
            return 1,2
        elif constant.r_head_boundary>r:
            return 1 ,1
        return 0,0
    else:
        if r>constant.r_gen_boundary and kwargs['last_boundary']==0:
            return 0 ,0
        elif r<constant.r_gen_boundary and kwargs['last_boundary']==0:
            return 1 ,1
        elif kwargs['last_boundary']==3:
            return kwargs['last_boundary'], 0
        elif theta_par_out_b2>kwargs['theta'] and kwargs['last_boundary']==1:
            return 2 ,1
        elif theta_par_in_b2<kwargs['theta'] and kwargs['last_boundary']==2:
            return 3 ,1
        return kwargs['last_boundary'], 0

#%%iteration to get next point
def point_equation(delta_theta, x, y, now_boundary, isboundary, d, equations):
    if now_boundary==0 or now_boundary==3:
        r=sqrt(x**2+y**2)
        if now_boundary==0:
            theta1=2*pi*r/constant.r0-pi
            theta2=theta1-delta_theta
            helice_out=equations['helice_out']
            x_new,y_new=helice_out.count_position_by_phase(theta2)
        elif now_boundary==3:
            theta1=2*pi*r/constant.r0
            theta2=theta1+delta_theta
            helice_in=equations['helice_in']
            x_new,y_new=helice_in.count_position_by_phase(theta2)

    elif now_boundary==1:
        if isboundary==1:
            circle_out=equations['circle_out']
            x_new,y_new=circle_out.count_position_by_phase(delta_theta)

    elif now_boundary==2:
        if isboundary==1:
            circle_in=equations['circle_in']
            x_new,y_new=circle_in.count_position_by_phase(delta_theta)
```

```
    distance=(x-x_new)**2+(y-y_new)**2
    return distance-d**2

def count_next_point_position( x, y, now_boundary, isboundary, equations, ishead,
constant):

    if now_boundary==0 or now_boundary==3:#迭代新点在螺旋线上
        if now_boundary==0:
            theta1=2*pi*constant.R/constant.r0
            helice_out=equations['helice_out']
            if ishead==1:
                delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
            else:
                delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))

x_next,y_next=helice_out.count_position_by_phase(theta1-delta_theta)

        elif now_boundary==3:
            theta1=2*pi*constant.R/constant.r0
            if isboundary==1:#边界三边界处理
                if ishead==1:
                    delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                else:
                    delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))
            else:
                if ishead==1:
                    delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                else:
                    delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))

x_next,y_next=helice_in.count_position_by_phase(theta1+delta_theta)
        return x_next ,y_next

    elif now_boundary==1:
        circle_out=equations['circle_out']
        if isboundary==1:#边界一边界处理
            if ishead==1:
```

```
                        delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                    else:
                        delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))
                    x_new,y_new=circle_out.count_position_by_phase(delta_theta)

        elif now_boundary==2:
            circle_in=equations['circle_in']
            if isboundary==1:#边界二边界处理
                if ishead==1:
                    delta_theta=fsolve(point_equation,
pi/2,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                else:
                    delta_theta=fsolve(point_equation,
pi/2,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))
                x_new,y_new=circle_in.count_position_by_phase(delta_theta)
        return x_new,y_new,delta_theta


def count_position_all(x_head , y_head, t, constant, equations):#(t>0)
        #得出每一点的位置(x,y,r)
        circle_out=equations['circle_out']
        circle_in=equations['circle_in']
        xs,ys=[x_head],[y_head]
        thetas_cir_in,thetas_cir_out=[],[]
        boundarys=[]
        positions_moment=[x_head,y_head]
        for i in range(constant.num_chair):
            x,y=xs[i],ys[i]
            if i==0:
                now_boundary,isboundary_head=judge_boundary(x,  y,  constant,  1,
t=t)
                if isboundary_head==1:
                    boundarys.append(now_boundary-1)
                else:
                    boundarys.append(now_boundary)
                if now_boundary==0 or now_boundary==3:
                    x_new,y_new=count_next_point_position(x,  y,  now_boundary,
isboundary_head, equations, 1, constant)
                else:#龙头在圆弧上
                    if isboundary_head==1:
                        x_new,y_new,delta_theta=count_next_point_position(x,   y,
now_boundary, isboundary_head, equations, 1, constant)
```

```
                    if now_boundary==1:
                        thetas_cir_out.append(delta_theta)
                    elif now_boundary==2:
                        thetas_cir_in.append(delta_theta)
                else:
                    if now_boundary==1:

theta0=constant.v*(t-constant.t1_l)/constant.r0+circle_in.phi0
                        theta_new=theta0-constant.gamma2_head

x_new,y_new=circle_out.count_position_by_phase(theta_new)
                        thetas_cir_out.append(theta_new)
                    elif now_boundary==2:
                        theta0=-constant.v*t/constant.r0+circle_out.phi0
                        theta_new=theta0+constant.gamma1_head

x_new,y_new=circle_in.count_position_by_phase(theta_new)
                        thetas_cir_in.append(theta_new)

        else:
            last_boundary=boundarys[-1]
            if last_boundary==0:
                now_boundary,isboundary=judge_boundary(x, y, constant, 0,
last_boundary=last_boundary)
            else:
                if last_boundary==1:
                    theta=thetas_cir_out[-1]
                    now_boundary,isboundary=judge_boundary(x, y, constant,
0, last_boundary=last_boundary ,theta=theta)
                elif last_boundary==2:
                    theta=thetas_cir_in[-1]
                    now_boundary,isboundary=judge_boundary(x, y, constant,
0, last_boundary=last_boundary ,theta=theta)
                elif last_boundary==3:
                    now_boundary,isboundary=judge_boundary(x, y, constant,
0, last_boundary=last_boundary)

            if now_boundary==0 or now_boundary==3:
                x_new,y_new=count_next_point_position(x, y, now_boundary,
isboundary, equations, 0, constant)
            else:
                if isboundary==1:
                    x_new,y_new,delta_theta=count_next_point_position(x, y,
now_boundary, isboundary, equations, 0, constant)
```

```
                        if now_boundary==1:
                            thetas_cir_out.append(delta_theta)
                        elif now_boundary==2:
                            thetas_cir_in.append(delta_theta)
                    else:
                        if now_boundary==1:
                            theta_new=thetas_cir_out[-1]-constant.gamma2_gen

x_new,y_new=circle_out.count_position_by_phase(theta_new)
                            thetas_cir_out.append(theta_new)
                        elif now_boundary==2:
                            theta_new=thetas_cir_in[-1]+constant.gamma1_gen

x_new,y_new=circle_in.count_position_by_phase(theta_new)
                            thetas_cir_in.append(theta_new)
            boundarys.append(now_boundary)
            xs.append(x_new)
            ys.append(y_new)
            positions_moment.append(x_new)
            positions_moment.append(y_new)
    def ensure_numeric(lst):
        lst_rebuild=[]
        for i in lst:
            if type(i)==float:
                lst_rebuild.append(float(i))
            elif type(i)=='numpy.float64':
                lst_rebuild.append(Decimal('numpy_float'))
        return lst_rebuild
    xs,ys,positions_moment                                                    =
ensure_numeric(xs),ensure_numeric(ys),ensure_numeric(positions_moment)
    return xs,ys,boundarys,positions_moment




def count_angle(xs,ys,boundarys,constant):
    #计算每条凳子前端与后端与相应切线或者竖直直线的夹角
    def linear_para(x1,y1,x2,y2):#得出一般直线方程系数
        return y1-y2,x2-x1,x1*y2-x2*y1
    angles_front,angles_behind,r0=[],[],constant.r0
    for i in range(constant.num_chair):
        x1,x2,y1,y2=xs[i],xs[i+1],ys[i],ys[i+1]
        A,B,C=linear_para(xs[i],ys[i],xs[i+1],ys[i+1])
        if boundarys[i]==0:
```

```
                r1=sqrt(x1**2+y1**2)
                r2=sqrt(x2**2+y2**2)
                if boundarys[i+1]==1:
                    alpha1=atan(1/(((2*pi*r1)/r0)+pi))
                    beta=atan((A*tan(2*pi*r1/r0)-B)/(A+B*tan(2*pi*r1/r0)))
                    angles_front.append(alpha1-beta)

alpha2=acos(abs(A*constant.x_Oout+B*constant.y_Oout+C)/(sqrt(A**2+B**2)*cons
tant.r2))
                    angles_behind.append(alpha2)
                else:
                    alpha1=atan(1/(((2*pi*r1)/r0)+pi))
                    alpha2=atan(1/(((2*pi*r1)/r0)+pi))
                    beta=atan((A*tan(2*pi*r1/r0)-B)/(A+B*tan(2*pi*r1/r0)))
                    angles_front.append(abs(alpha1-beta))
                    angles_behind.append(abs(alpha2-beta))
            elif boundarys[i]==1:
                if boundarys[i+1]==1:
                    angles_front.append(0)
                    angles_behind.append(0)
                else:

alpha1=acos(abs(A*constant.x_Oout+B*constant.y_Oout+C)/(sqrt(A**2+B**2)*cons
tant.r2))

alpha2=acos(abs(A*constant.x_Oin+B*constant.y_Oin+C)/(sqrt(A**2+B**2)*constan
t.r1))
                    angles_front.append(alpha1)
                    angles_behind.append(alpha2)
            elif boundarys[i]==2:
                if boundarys[i+1]==2:
                    angles_front.append(0)
                    angles_behind.append(0)
                else:

alpha1=acos(abs(A*constant.x_Oin+B*constant.y_Oin+C)/(sqrt(A**2+B**2)*constan
t.r1))
                    alpha2=atan(r0/(2*pi*r2))
                    beta2=atan((A*tan(2*pi*r2/r0)-B)/(A+B*tan(2*pi*r2/r0)))
                    angles_front.append(alpha1)
                    angles_behind.append(alpha2-beta2)
            elif boundarys[i]==3:
                alpha1=atan(r0/(2*pi*r1))
                alpha2=atan(r0/(2*pi*r2))
```

```
                beta1=atan((A*tan(2*pi*r1/r0)-B)/(A+B*tan(2*pi*r1/r0)))
                beta2=atan((A*tan(2*pi*r2/r0)-B)/(A+B*tan(2*pi*r2/r0)))
                angles_front.append(abs(alpha1-beta1))
                angles_behind.append(abs(alpha2-beta2))
    return angles_front,angles_behind

def count_velocity_all(angles_front,angles_behind):
    velocitys=[1]
    for i in range(constant.num_chair):
        velocity_new=velocitys[i]*cos(angles_front[i])/cos(angles_behind[i])
        velocitys.append(velocity_new)
    return velocitys

#%%main_code
constant=figurable_constant(constant)
circle_in=circle_equation(constant.theta0-constant.alpha, constant.r1, -constant.v,
constant.x_Oin, constant.y_Oin, constant.t1_l)
circle_out=circle_equation(constant.theta0-pi-constant.gamma-constant.alpha,
constant.r2, constant.v, constant.x_Oout, constant.y_Oout ,constant.t1_l)
helice_in=helice(constant.r0,0)
helice_out=helice(constant.r0,pi)
equations={'circle_in':circle_in,'circle_out':circle_out,'helice_in':helice_in,'helice_out'
:helice_out}
x_head,y_head=circle_in.count_position_by_time(0)
xs,ys,boundarys,positions=count_position_all(x_head , y_head, 0, constant,
equations)

'''
t_prob4=100
x_moments,y_moments,boundarys_moments,positions_moments=[],[],[],[]
for t in range(1,t_prob4+1):
    if t<=11:
        x_head,y_head=circle_in.count_position_by_time(t)
        xs,ys,boundarys,positions=count_position_all(x_head , y_head, t, constant,
equations)
    else:
        x_head,y_head=x_head_position2[t-12],y_head_position2[t-12]
        xs,ys,boundarys,positions=count_position_all(x_head, y_head, t, constant,
equations)
    x_moments.append(xs)
    y_moments.append(ys)
    boundarys_moments.append(boundarys)
    positions_moments.append(positions)
```

```
velocitys_moments=[]
for t in range(1,t_prob4):
    xs,ys,boundary=x_moments[t],y_moments[t],boundarys_moments[t]
    angles_front,angles_behind=count_angle(xs, ys, boundarys, constant)
    velocitys=count_velocity_all(angles_front, angles_behind)
    velocitys.moments.append(velocitys)

positions=positions_moments
from openpyxl import load_workbook
positions = np.array(positions)
velocitys = np.array(velocitys)
positions = positions.T
velocitys = velocitys.T
file_path = "C:/Users/AlbertYang/Desktop/2024 国赛/A 题/附件/result4.xlsx"
wb = load_workbook(file_path)
start_row = 2
start_col = 2
ws = wb.active
ws1 = wb['位置']
ws2 = wb['速度']
for i, row in enumerate(positions):
    for j, value in enumerate(row):
        ws1.cell(row=start_row + i, column=start_col + j, value=value)
for i, row in enumerate(velocitys):
    for j, value in enumerate(row):
        ws2.cell(row=start_row + i, column=start_col + j, value=value)
wb.save(file_path)

'''
```

第五问代码

```
import numpy as np
import math
from math import pi
import seaborn as sns
import matplotlib.pyplot as plt
sns.set()
from math import pi, log, sqrt, cos, sin,atan
import numpy as np
from scipy.optimize import fsolve
import matplotlib.pyplot as plt
from math import pi,sqrt,cos,sin,atan,tan,asin,acos
from scipy.optimize import fsolve
```

```
import numpy as np
from decimal import Decimal
from prob1 import constant
from prob4_head_positions2 import x_head_position2,y_head_position2
#from prob4_calc_r_boundary import r_boundary

def cot(angle):
	return 1/tan(angle)

class figurable_constant(object):
	#所有初始常量以及通过手工数学计算能算出的恒量
	def __init__(self,constant,v):
		self.d_head=constant.d_head
		self.d_gen=constant.d_gen
		self.num_chair=constant.num_chair
		self.r_head_boundary=4.67131999
		self.r_gen_boundary=4.67131999
		self.R=4.5
		self.r0=1.7
		self.v=v
		self.theta0=2*pi*self.R/self.r0-4*pi
		self.alpha=atan(self.R/(2*pi*self.r0))
		self.x_in=self.R*cos(self.theta0)
		self.y_in=self.R*sin(self.theta0)

self.d_l1=abs(self.y_in-self.x_in*cot(self.theta0-self.alpha))/sqrt(1+cot(self.alpha-self.t
heta0)**2)

self.d_l2=abs(self.y_in-self.x_in*tan(self.theta0-pi-self.alpha))/sqrt(1+tan(self.alpha-se
lf.theta0)**2)
		self.l1=2*self.d_l2
		self.l2=2*self.d_l1
		self.r1=(self.l1**2+self.l2**2)/(3*self.l2)
		self.r2=self.r1/2
		self.gamma=pi-asin(2*self.l1*self.l2/(self.l1**2+self.l2**2))
		self.gamma1_head=2*asin(self.d_head/(2*self.r1))
		self.gamma2_head=2*asin(self.d_head/(2*self.r2))
		self.gamma1_gen=2*asin(self.d_gen/(2*self.r1))
		self.gamma2_gen=2*asin(self.d_gen/(2*self.r2))
		self.t1_head=self.gamma1_head*self.r1/self.v
		self.t2_head=self.gamma2_head*self.r2/self.v
		self.t1_gen=self.gamma1_gen*self.r1/self.v
		self.t2_gen=self.gamma2_gen*self.r2/self.v
		self.t1_l=self.gamma*self.r1/self.v
```

```
        self.t2_l=self.gamma*self.r2/self.v
        self.x_Oin=self.x_in+self.r1/sqrt(1+tan(self.theta0-self.alpha)**2)

self.y_Oin=self.y_in+self.r1*tan(self.theta0-pi-self.alpha)/sqrt(1+tan(self.theta0-pi-self
.alpha)**2)
        self.x_Oout=self.x_in-self.r1/sqrt(1+tan(self.theta0-self.alpha)**2)

self.y_Oout=self.y_in-self.r1*tan(self.theta0-pi-self.alpha)/sqrt(1+tan(self.theta0-pi-se
lf.alpha)**2)
        self.t_sum_turn_back=self.gamma*(self.r1+self.r2)/self.v

class circle_equation(object):
    #圆的参数方程对象
    def __init__(self,phi0,radius,velocity,x_O,y_O,t_center):
        self.phi0=phi0
        self.radius=radius
        self.velocity=velocity
        self.x_O=x_O
        self.y_O=y_O
        self.t_center=t_center

    def count_position_by_time(self,t):
        if t<=self.t_center:
            pass
        else:
            t=t-self.t_center
        x=self.radius*cos(self.radius*t/self.velocity+self.phi0)+self.x_O
        y=self.radius*sin(self.radius*t/self.velocity+self.phi0)+self.y_O
        return x,y

    def count_position_by_phase(self,phase):
        x=self.radius*cos(phase)+self.x_O
        y=self.radius*sin(phase)+self.y_O
        return x,y

class helice(object):
    def __init__(self,r0,phi0):
        self.r0=r0
        self.phi0=phi0

    def count_position_by_phase(self,phase):
        x=self.r0*(phase+self.phi0)*cos(phase)/(2*pi)
        y=self.r0*(phase+self.phi0)*sin(phase)/(2*pi)
        return x,y
```

```
#%%judge boundary condition
def judge_boundary(xi ,yi, constant, ishead, **kwargs):
    #theta_par_in_b3=constant.theta0-constant.alpha
    theta_par_in_b2=-constant.alpha+(pi-constant.theta0)+(pi-constant.gamma)
    theta_par_out_b2=constant.theta0-pi-constant.gamma-constant.alpha
    #theta_par_out_b1=constant.theta0-constant.alpha+constant.gamma
    r=sqrt(xi**2+yi**2)
    if ishead==1:
        if kwargs['t']==0:
            return 3 ,2
        elif 0<kwargs['t'] and kwargs['t']<=constant.t1_l:
            if kwargs['t']<constant.t1_head:
                return 3 ,1
            return 2 ,2
        elif constant.t1_l<kwargs['t'] and kwargs['t']-constant.t1_l<=constant.t2_l:
            if kwargs['t']-constant.t1_l<constant.t2_head:
                return 2 ,1
            return 1,2
        elif constant.r_head_boundary>r:
            return 1 ,1
        return 0,0
    else:
        if r>constant.r_gen_boundary and kwargs['last_boundary']==0:
            return 0 ,0
        elif r<constant.r_gen_boundary and kwargs['last_boundary']==0:
            return 1 ,1
        elif kwargs['last_boundary']==3:
            return kwargs['last_boundary'], 0
        elif theta_par_out_b2>kwargs['theta'] and kwargs['last_boundary']==1:
            return 2 ,1
        elif theta_par_in_b2<kwargs['theta'] and kwargs['last_boundary']==2:
            return 3 ,1
        return kwargs['last_boundary'], 0

#%%iteration to get next point
def point_equation(delta_theta, x, y, now_boundary, isboundary, d, equations):
    if now_boundary==0 or now_boundary==3:
        r=sqrt(x**2+y**2)
        if now_boundary==0:
            theta1=2*pi*r/constant.r0-pi
            theta2=theta1-delta_theta
            helice_out=equations['helice_out']
            x_new,y_new=helice_out.count_position_by_phase(theta2)
```

```
            elif now_boundary==3:
                theta1=2*pi*r/constant.r0
                theta2=theta1+delta_theta
                helice_in=equations['helice_in']
                x_new,y_new=helice_in.count_position_by_phase(theta2)

        elif now_boundary==1:
            if isboundary==1:
                circle_out=equations['circle_out']
                x_new,y_new=circle_out.count_position_by_phase(delta_theta)

        elif now_boundary==2:
            if isboundary==1:
                circle_in=equations['circle_in']
                x_new,y_new=circle_in.count_position_by_phase(delta_theta)

        distance=(x-x_new)**2+(y-y_new)**2
        return distance-d**2

def count_next_point_position( x, y, now_boundary, isboundary, equations, ishead,
constant):

        if now_boundary==0 or now_boundary==3:#迭代新点在螺旋线上
            if now_boundary==0:
                theta1=2*pi*constant.R/constant.r0
                helice_out=equations['helice_out']
                if ishead==1:
                    delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                else:
                    delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))

x_next,y_next=helice_out.count_position_by_phase(theta1-delta_theta)

            elif now_boundary==3:
                theta1=2*pi*constant.R/constant.r0
                if isboundary==1:#边界三边界处理
                    if ishead==1:
                        delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                    else:
                        delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))
```

```
                else:
                    if ishead==1:
                        delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                    else:
                        delta_theta=fsolve(point_equation,
pi/12,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))

x_next,y_next=helice_in.count_position_by_phase(theta1+delta_theta)
            return x_next ,y_next

        elif now_boundary==1:
            circle_out=equations['circle_out']
            if isboundary==1:#边界一边界处理
                if ishead==1:
                    delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                else:
                    delta_theta=fsolve(point_equation,
0,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))
                x_new,y_new=circle_out.count_position_by_phase(delta_theta)

        elif now_boundary==2:
            circle_in=equations['circle_in']
            if isboundary==1:#边界二边界处理
                if ishead==1:
                    delta_theta=fsolve(point_equation,
pi/2,args=(x,y,now_boundary,isboundary,constant.d_head,equations))
                else:
                    delta_theta=fsolve(point_equation,
pi/2,args=(x,y,now_boundary,isboundary,constant.d_gen,equations))
                x_new,y_new=circle_in.count_position_by_phase(delta_theta)
        return x_new,y_new,delta_theta


def count_position_all(x_head , y_head, t, constant, equations):#(t>0)
        #得出每一点的位置(x,y,r)
        circle_out=equations['circle_out']
        circle_in=equations['circle_in']
        xs,ys=[x_head],[y_head]
        thetas_cir_in,thetas_cir_out=[],[]
        boundarys=[]
        positions_moment=[x_head,y_head]
        for i in range(constant.num_chair):
```

```
        x,y=xs[i],ys[i]
        if i==0:
            now_boundary,isboundary_head=judge_boundary(x, y, constant, 1,
t=t)
            if isboundary_head==1:
                boundarys.append(now_boundary-1)
            else:
                boundarys.append(now_boundary)
            if now_boundary==0 or now_boundary==3:
                x_new,y_new=count_next_point_position(x, y, now_boundary,
isboundary_head, equations, 1, constant)
            else:#龙头在圆弧上
                if isboundary_head==1:
                    x_new,y_new,delta_theta=count_next_point_position(x, y,
now_boundary, isboundary_head, equations, 1, constant)
                    if now_boundary==1:
                        thetas_cir_out.append(delta_theta)
                    elif now_boundary==2:
                        thetas_cir_in.append(delta_theta)
                else:
                    if now_boundary==1:

theta0=constant.v*(t-constant.t1_l)/constant.r0+circle_in.phi0
                        theta_new=theta0-constant.gamma2_head

x_new,y_new=circle_out.count_position_by_phase(theta_new)
                        thetas_cir_out.append(theta_new)
                    elif now_boundary==2:
                        theta0=-constant.v*t/constant.r0+circle_out.phi0
                        theta_new=theta0+constant.gamma1_head

x_new,y_new=circle_in.count_position_by_phase(theta_new)
                        thetas_cir_in.append(theta_new)

        else:
            last_boundary=boundarys[-1]
            if last_boundary==0:
                now_boundary,isboundary=judge_boundary(x, y, constant, 0,
last_boundary=last_boundary)
            else:
                if last_boundary==1:
                    theta=thetas_cir_out[-1]
                    now_boundary,isboundary=judge_boundary(x, y, constant,
0, last_boundary=last_boundary ,theta=theta)
```

```
                        elif last_boundary==2:
                                theta=thetas_cir_in[-1]
                                now_boundary,isboundary=judge_boundary(x,  y,  constant,
0, last_boundary=last_boundary ,theta=theta)
                        elif last_boundary==3:
                                now_boundary,isboundary=judge_boundary(x,  y,  constant,
0, last_boundary=last_boundary)

                if now_boundary==0 or now_boundary==3:
                        x_new,y_new=count_next_point_position(x,   y,   now_boundary,
isboundary, equations, 0, constant)
                else:
                        if isboundary==1:
                                x_new,y_new,delta_theta=count_next_point_position(x,    y,
now_boundary, isboundary, equations, 0, constant)
                                if now_boundary==1:
                                        thetas_cir_out.append(delta_theta)
                                elif now_boundary==2:
                                        thetas_cir_in.append(delta_theta)
                        else:
                                if now_boundary==1:
                                        theta_new=thetas_cir_out[-1]-constant.gamma2_gen

x_new,y_new=circle_out.count_position_by_phase(theta_new)
                                        thetas_cir_out.append(theta_new)
                                elif now_boundary==2:
                                        theta_new=thetas_cir_in[-1]+constant.gamma1_gen

x_new,y_new=circle_in.count_position_by_phase(theta_new)
                                        thetas_cir_in.append(theta_new)
            boundarys.append(now_boundary)
            xs.append(x_new)
            ys.append(y_new)
            positions_moment.append(x_new)
            positions_moment.append(y_new)
    def ensure_numeric(lst):
            lst_rebuild=[]
            for i in lst:
                    if type(i)==float:
                            lst_rebuild.append(float(i))
                    elif type(i)=='numpy.float64':
                            lst_rebuild.append(Decimal('numpy_float'))
            return lst_rebuild
    xs,ys,positions_moment                                                          =
```

```
ensure_numeric(xs),ensure_numeric(ys),ensure_numeric(positions_moment)
    return xs,ys,boundarys,positions_moment

#%%计算速度


def count_angle(xs,ys,boundarys,constant):
    #计算每条凳子前端与后端与相应切线或者竖直直线的夹角
    def linear_para(x1,y1,x2,y2):#得出一般直线方程系数
        return y1-y2,x2-x1,x1*y2-x2*y1
    angles_front,angles_behind,r0=[],[],constant.r0
    for i in range(constant.num_chair):
        x1,x2,y1,y2=xs[i],xs[i+1],ys[i],ys[i+1]
        A,B,C=linear_para(xs[i],ys[i],xs[i+1],ys[i+1])
        if boundarys[i]==0:
            r1=sqrt(x1**2+y1**2)
            r2=sqrt(x2**2+y2**2)
            if boundarys[i+1]==1:
                alpha1=atan(1/(((2*pi*r1)/r0)+pi))
                beta=atan((A*tan(2*pi*r1/r0)-B)/(A+B*tan(2*pi*r1/r0)))
                angles_front.append(alpha1-beta)

alpha2=acos(abs(A*constant.x_Oout+B*constant.y_Oout+C)/(sqrt(A**2+B**2)*cons
tant.r2))
                angles_behind.append(alpha2)
            else:
                alpha1=atan(1/(((2*pi*r1)/r0)+pi))
                alpha2=atan(1/(((2*pi*r1)/r0)+pi))
                beta=atan((A*tan(2*pi*r1/r0)-B)/(A+B*tan(2*pi*r1/r0)))
                angles_front.append(abs(alpha1-beta))
                angles_behind.append(abs(alpha2-beta))
        elif boundarys[i]==1:
            if boundarys[i+1]==1:
                angles_front.append(0)
                angles_behind.append(0)
            else:

alpha1=acos(abs(A*constant.x_Oout+B*constant.y_Oout+C)/(sqrt(A**2+B**2)*cons
tant.r2))

alpha2=acos(abs(A*constant.x_Oin+B*constant.y_Oin+C)/(sqrt(A**2+B**2)*constan
t.r1))
                angles_front.append(alpha1)
                angles_behind.append(alpha2)
```

```
            elif boundarys[i]==2:
                if boundarys[i+1]==2:
                    angles_front.append(0)
                    angles_behind.append(0)
                else:

alpha1=acos(abs(A*constant.x_Oin+B*constant.y_Oin+C)/(sqrt(A**2+B**2)*constan
t.r1))
                    alpha2=atan(r0/(2*pi*r2))
                    beta2=atan((A*tan(2*pi*r2/r0)-B)/(A+B*tan(2*pi*r2/r0)))
                    angles_front.append(alpha1)
                    angles_behind.append(alpha2-beta2)
            elif boundarys[i]==3:
                alpha1=atan(r0/(2*pi*r1))
                alpha2=atan(r0/(2*pi*r2))
                beta1=atan((A*tan(2*pi*r1/r0)-B)/(A+B*tan(2*pi*r1/r0)))
                beta2=atan((A*tan(2*pi*r2/r0)-B)/(A+B*tan(2*pi*r2/r0)))
                angles_front.append(abs(alpha1-beta1))
                angles_behind.append(abs(alpha2-beta2))
        return angles_front,angles_behind

def count_velocity_all(angles_front,angles_behind):
        velocitys=[1]
        for i in range(constant.num_chair):
            velocity_new=velocitys[i]*cos(angles_front[i])/cos(angles_behind[i])
            velocitys.append(velocity_new)
        return velocitys

#%%main_code
v=1
constant=figurable_constant(constant,v)#传递变量参数
t_pan=constant.t_sum_turn_back
t_frist=int(t_pan)+1#判断时间的标准，进行判别什么时候输入什么参数
def dh(tt,v):#定义并且寻找进入盘出螺线的各个时刻龙头位置
        k = 1.7/ (2 * pi)
        C = 0.5 * k**2 * log(k)

        def theta(r):
            return (r / k)-pi

        def r_(theta_):
            return k * (theta_+pi)
        rB = 4.5
        tB = (0.5 * (rB * sqrt(rB**2 + k**2) + k**2 * log(rB + sqrt(rB**2 + k**2))) - C) /k
```

```
*v
    t_out= t_pan#!!!
    t = tB+tt-t_out
    def equation(r, t=t):
        return 0.5 * (r * sqrt(r**2 + k**2) + k**2 * log(r + sqrt(r**2 + k**2))) - k
*v* t - C
    initial_guess = 1
    solution = fsolve(equation, initial_guess)
    r = solution[0]
    a = theta(r)
    x0 = r * cos(a)
    y0 = r * sin(a)
    return x0,y0
def head(t,v):#求解出任意时刻，任意速度的头位置
    if t>=t_pan:
        x_head,y_head=dh(t,v)
        return x_head,y_head
    else:
        #
        circle_in=circle_equation(constant.theta0-constant.alpha,        constant.r1,
-constant.v, constant.x_Oin, constant.y_Oin, constant.t1_l)

circle_out=circle_equation(constant.theta0-pi-constant.gamma-constant.alpha,
constant.r2, constant.v, constant.x_Oout, constant.y_Oout ,constant.t1_l)
        helice_in=helice(constant.r0,0)
        helice_out=helice(constant.r0,pi)

equations={'circle_in':circle_in,'circle_out':circle_out,'helice_in':helice_in,'helice_out'
:helice_out}
        x_head,y_head=circle_in.count_position_by_time(t)
        #
        return x_head,y_head
def max_v_idex(t,v):
    #
    x_head,y_head=head(t,v)
    circle_in=circle_equation(constant.theta0-constant.alpha,        constant.r1,
-constant.v, constant.x_Oin, constant.y_Oin, constant.t1_l)
    circle_out=circle_equation(constant.theta0-pi-constant.gamma-constant.alpha,
constant.r2, constant.v, constant.x_Oout, constant.y_Oout ,constant.t1_l)
    helice_in=helice(constant.r0,0)
    helice_out=helice(constant.r0,pi)

equations={'circle_in':circle_in,'circle_out':circle_out,'helice_in':helice_in,'helice_out'
:helice_out}
```

```
    x_head,y_head=circle_in.count_position_by_time(t)
    #
    xs,ys,boundarys,positions=count_position_all(x_head , y_head, 0, constant,
equations)
    b=boundarys
    idex=[]
    if b[0]==0:
        for i in range(len(b)):
            if b[i]==0 and b[i+1]==1:
                idex.append(i)
                idex.append(i+1)
            elif b[i]==1 and b[i+1]==2:
                idex.append(i+1)
            elif b[i]==2 and b[i+1]==3:
                idex.append(i+1)
                break
    elif b[0]==1:
        for i in range(len(b)):
            if b[i]==1 and b[i+1]==2:
                idex.append(i)
                idex.append(i+1)
            elif b[i]==2 and b[i+1]==3:
                idex.append(i+1)
    elif b[0]==2:
        for i in range(len(b)):
            if b[i]==2 and b[i+1]==3:
                idex.append(i)
                idex.append(i+1)
    return idex
def max_v(t,v):
    maxv=[]
    vv=count_velocity_all(angles_front,angles_behind)
    for i in max_v_idex(t,v):
        maxv.append(vv[i])
    return max(maxv)









for i in range(10):
    vv=[]
```

```
    dv=0.1
    dt=10#!!!
    for i in np.arange(0,101,dt):
        vv.append(max_v(i,v))
    max__v=max(vv)
    if max__v>2:
        print('速度为',v-dv,'时没超',v,'时超速了')
        break
    else:
        v+=dv
print('最大龙头速度为',v)




'''
circle_in=circle_equation(constant.theta0-constant.alpha, constant.r1, -constant.v, constant.x_Oin, constant.y_Oin, constant.t1_l)
circle_out=circle_equation(constant.theta0-pi-constant.gamma-constant.alpha, constant.r2, constant.v, constant.x_Oout, constant.y_Oout ,constant.t1_l)
helice_in=helice(constant.r0,0)
helice_out=helice(constant.r0,pi)
equations={'circle_in':circle_in,'circle_out':circle_out,'helice_in':helice_in,'helice_out':helice_out}
x_head,y_head=circle_in.count_position_by_time(1)
xs,ys,boundarys,positions=count_position_all(x_head , y_head, 0, constant, equations)
vs=count_velocity_all(count_angle(xs,ys,boundarys,constant))
'''
```